DFSha Informe técnico final

Integrantes: Nicolás Agudelo Mesa, Maria Alejandra Vargas Duque, David Alexander Salazar Villa

Docente: Alvaro Ospina

Sistemas Distribuidos

Universidad Pontificia Bolivariana

2026-2

1. Introducción

1.1 Contexto y descripción de DFSha

DFSha es un sistema de archivos distribuido diseñado para permitir el almacenamiento, la administración y el acceso a archivos mediante un conjunto de nodos conectados a través de una red. Distribuyendo los bloques que componen los archivos entre diferentes nodos, incorporando mecanismos de replicación para mejorar la disponibilidad y la tolerancia ante fallos.

El sistema adopta una arquitectura cliente/servidor con servicios distribuidos entre servidores (S2S). En esta arquitectura, los clientes interactúan con los servidores de metadatos para realizar operaciones de administración y localización de archivos, mientras que los nodos de datos se encargan de almacenar, transferir y replicar los bloques de información.

La comunicación entre los componentes se realiza mediante gRPC, utilizando interfaces de servicio definidas para intercambiar solicitudes y respuestas de manera estructurada. La implementación se desarrolla en Python y utiliza contenedores Docker para facilitar la configuración, ejecución y despliegue de los diferentes componentes del sistema.

De esta manera, DFSha busca proporcionar un servicio de almacenamiento distribuido que permita gestionar archivos de forma transparente para el usuario, manteniendo la integridad de la información y ofreciendo mecanismos de disponibilidad, consistencia, concurrencia y recuperación ante fallos.

1.2 Objetivo general

Diseñar, implementar y validar un sistema de archivos distribuido DFSha, basado en una arquitectura cliente/servidor con servicios distribuidos entre servidores, que permita almacenar, administrar, transferir y acceder a archivos mediante múltiples nodos. Con mecanismos de particionamiento, replicación, consistencia, tolerancia a fallos y seguridad,con el propósito de ofrecer un servicio de almacenamiento confiable, escalable y verificable.

1.3 Alcance del sistema

- **Gestión del sistema de archivos:** creación, eliminación y listado de directorios y archivos, así como la administración de los espacios de almacenamiento asociados a los usuarios.

- **Transferencia de archivos:** carga y descarga de archivos completos entre el cliente y la infraestructura distribuida.
- **Acceso a archivos:** implementación de operaciones como apertura, lectura, escritura y cierre de archivos, junto con mecanismos de control de acceso concurrente.
- **Distribución de información:** división de archivos en bloques y distribución de estos entre diferentes nodos de datos.
- **Replicación:** generación y mantenimiento de copias de los bloques para reducir el impacto de fallos en los nodos.
- **Administración de metadatos:** registro de información relacionada con archivos, directorios, versiones, ubicación de bloques y estado de las operaciones.
- **Consistencia e integridad:** incorporación de mecanismos que permitan verificar la integridad de los bloques y establecer condiciones de confirmación y visibilidad de las escrituras.
- **Tolerancia a fallos:** detección de nodos no disponibles y mecanismos de recuperación o utilización de réplicas cuando sea posible.
- **Seguridad:** autenticación de usuarios, control de permisos y protección de las comunicaciones entre los componentes.

1.4 Supuestos y limitaciones

Para el diseño se establecen los siguientes supuestos:

- Los nodos se comunican mediante una red privada y cuentan con conectividad entre los componentes autorizados.
- Cada usuario dispone de un espacio lógico de almacenamiento independiente.
- Los nodos de metadatos mantienen la información necesaria para localizar los archivos y sus bloques.
- Los nodos de datos pueden fallar de manera independiente, por lo que el sistema debe mantener réplicas de la información.
- La disponibilidad de las operaciones que modifican metadatos depende de que exista una mayoría de servidores Raft.

Se reconocen las siguientes limitaciones de diseño:

- No se busca implementar toda la semántica de un sistema de archivos POSIX.
- La implementación de cifrado, gestión de claves y autenticación estará limitada a las capacidades y recursos disponibles en el entorno académico.

2. Requisitos y funcionalidades

2.1 Requisitos funcionales

RF1: Gestión del sistema de archivos
El sistema permitirá que los usuarios autenticados administren su espacio lógico de almacenamiento independiente mediante operaciones de creación, consulta y eliminación de archivos y directorios.

Por defecto, solamente podrá consultar y modificar los archivos sobre los cuales tenga autorización.

RF2:Transferencia de archivos completos

El sistema permitirá transferir archivos completos entre el cliente y DFSha.

Durante la carga, el cliente dividirá el archivo en bloques y enviará los bloques a los nodos de datos asignados. Cada bloque se replicará en nodos diferentes. Durante la recuperación, el cliente consultará los metadatos para obtener la ubicación de los bloques y descargará las partes necesarias para reconstruir el archivo original.

RF3: Acceso a archivos

El sistema proporcionará operaciones para acceder y modificar archivos mediante una interfaz de lectura y escritura.

La implementación deberá respetar la semántica definida en la sección 2.4 y garantizar que una escritura incompleta no sea visible como una versión confirmada del archivo.

2.2 Requisitos no funcionales

RNF1-Escalabilidad: permitir aumentar progresivamente la cantidad de usuarios, archivos y nodos de almacenamiento sin modificar la interfaz lógica que utiliza el cliente.

RNF2-Alta disponibilidad: mantener el acceso a la información cuando alguno de los nodos de datos falle, siempre que existan réplicas disponibles. El clúster de metadatos deberá tolerar la caída de uno de sus tres servidores.

RNF3-Consistencia: garantizar que las escrituras confirmadas se publiquen de manera coordinada y que las lecturas no accedan a versiones incompletas o no confirmadas.

RNF4-Particionamiento: distribuir los bloques de un mismo archivo entre varios nodos de datos, evitando que todo el contenido dependa de un único servidor.

RNF5-Rendimiento y concurrencia: permitir transferencias mediante streaming, lecturas desde diferentes nodos y acceso concurrente controlado a los archivos.

RNF6-Seguridad: incorporar autenticación, autorización, cifrado en tránsito y protección de la información almacenada.

RNF7-Transparencia de acceso y localización: evitar que el cliente dependa de direcciones físicas codificadas de los nodos. Las ubicaciones deberán obtenerse dinámicamente a través de los servidores de metadatos.

2.3 Operaciones y casos de uso

Las operaciones y los casos de uso para cada requisito funcional son:

FR1:

- ls: listar el contenido de un directorio.
- cd: cambiar el directorio actual del cliente.
- mkdir: crear un directorio.
- rmdir: eliminar un directorio vacío.
- rm: eliminar un archivo.
- Pwd: para consultar y administrar rutas.

RF2**:**

- put: cargar un archivo desde el cliente hacia el sistema distribuido.
- get: recuperar un archivo almacenado en DFSha hacia el cliente.

FR3:

- open(): abrir un archivo y obtener una sesión de acceso.
- read(): leer el contenido de un archivo o una porción del flujo disponible.
- write(): escribir información mediante la creación, reemplazo o anexado de contenido.
- close(): cerrar la sesión y confirmar los cambios pendientes.
- lock(): solicitar un bloqueo exclusivo para coordinar operaciones concurrentes.
- unlock(): liberar el bloqueo adquirido.

2.4 Semántica acordada de read() y write()

La operación read() permitirá recuperar el contenido de un archivo a partir de los bloques que lo componen. El cliente consultará los metadatos para conocer las ubicaciones de las réplicas y solicitará los bloques a los nodos de datos disponibles. Si un nodo no responde o presenta un error de integridad, el cliente podrá intentar recuperar el mismo bloque desde otra réplica.

La operación write() admitirá tres modalidades:

1. **Crear:** generar un archivo nuevo y almacenar su contenido en bloques distribuidos.
2. **Reemplazar:** generar una nueva versión del archivo sin modificar directamente la versión confirmada anteriormente.
3. **Anexar:** agregar contenido al final del archivo, coordinando la operación mediante un bloqueo exclusivo.

No se implementará modificación parcial arbitraria dentro de un bloque. Esta decisión reduce la complejidad de la sincronización y de la replicación, y permite mantener una política de publicación de versiones más sencilla de verificar.

Los cambios permanecerán en estado temporal mientras se transfieren y replican los bloques. Una vez que se cumpla el criterio de confirmación, el sistema publicará la nueva versión del archivo mediante una operación de confirmación coordinada con los servidores de metadatos.

Las lecturas nuevas accederán a la última versión confirmada. Las escrituras que no hayan completado el proceso de confirmación no deberán aparecer como archivos definitivos

3. Comparación de las propuestas individuales

3.1 Matriz comparativa de propuestas

Decisión	David	Nicolás	María	Decisión final
Arquitectura	**S2S**	**S2S**	**S2S**	**S2S, opción 1**
Servidores de metadatos	**3**	**3**	**3**	**3**
Consenso de metadatos	**Raft**	**Raft**	**Raft**	**Raft (PySyncObj)**

Nodos de datos	**5**	**4**	**5**	**5**
Tamaño de bloque	**8 MiB**	**8 MiB**	**64 MiB**	**8 MiB**
Factor de replicación	**3**	**3**	**3**	**3**
Quórum de escritura	**2 de 3**	**2 de 3**	**2 de 3**	**2 de 3**
Transporte de bloques	**gRPC streaming**	**gRPC de control y TCP/TLS para los datos**	**gRPC y un canal de datos separado**	**gRPC streaming en los dos planos**
Integridad	**SHA-256**	**SHA-256**	**Checksums por segmentos**	**SHA-256 del bloque, más el tag de AES-GCM**
Concurrencia	**Lock exclusivo y lease**	**Locks compartido y exclusivo, y lease**	**Leases, versiones y copy-on-write**	**Locks S y X, lease de 30 s y versión**
Edición dentro de un bloque	**No**	**No**	**Sí, con copy-on-write**	**No. Crear, reemplazar y anexar**

Detección de fallos	**Heartbeats**	**Heartbeats**	**Heartbeats y reportes de bloques**	**Heartbeat cada 3 s y reporte al arrancar y cada hora**
Cifrado	**No detallado**	**El cliente cifra y el nodo no ve el claro**	**No detallado**	**AES-256-GCM en el nodo de datos**
Despliegue	**AWS, red privada**	**Una VM por proceso**	**AWS y Docker**	**Docker Compose en una VM de AWS Academy**

3.1 Criterios utilizados para seleccionar la arquitectura

La selección de la arquitectura final se fundamenta en los siguientes criterios:

- Cumplimiento del enunciado
- Simplicidad: se priorizan mecanismos que permitan demostrar las propiedades del sistema sin introducir complejidad innecesaria.
- Consistencia: las operaciones confirmadas deben tener un comportamiento definido y verificable.
- Tolerancia a fallos: la pérdida de un nodo no debe implicar necesariamente la pérdida de los datos.
- Escalabilidad: la arquitectura debe permitir agregar nodos de almacenamiento y aumentar la capacidad del sistema.
- Mantenibilidad: se busca reducir la cantidad de protocolos y componentes

3.2 Decisiones adoptadas, descartadas y modificadas

La arquitectura final es la opción 1, cliente/servidor con el servicio distribuido entre servidores. Los tres diseños ya partían de esa opción, de tres servidores de metadatos con Raft, de tres réplicas y de un quórum de escritura de dos. Esas cuatro decisiones se conservaron porque cumplen el enunciado y porque coincidían. Donde los diseños diferían, el equipo eligió la variante que se podía implementar y comprobar en AWS Academy.

Cinco nodos de datos. Se tomó el número de David y de María. Con tres réplicas obligatorias en nodos distintos, cuatro nodos, como proponía Nicolás, alcanzan mientras todos estén vivos. Al caer uno quedan tres, justo el factor de replicación, y no hay un nodo libre para reponer la copia. Cinco dejan un margen para esa reposición.

Bloques de 8 MiB. Se tomó el tamaño de David y de Nicolás, y se descartaron los 64 MiB de María. Un archivo de 20 MB ya produce tres bloques y obliga a usar varios nodos, que es lo que hay que demostrar. Con 64 MiB haría falta un archivo mucho mayor para ver el mismo efecto, y el disco de la VM de prueba era de 8 GB.

Un solo transporte, gRPC. Se tomó el streaming de David. El cliente habla con los metadatos y con los nodos de datos por servicios distintos, así que el plano de control y el plano de datos siguen separados, que era la intención del canal aparte de María y del TCP aparte de Nicolás. Un segundo protocolo habría duplicado el cifrado en tránsito y los certificados. gRPC ya trae el streaming de los bloques.

SHA-256 por bloque. Se tomó la integridad de David y de Nicolás. El checksum por segmentos de María detecta cortes, pero el hash del bloque completo es el que el cliente compara al armar el archivo. AES-256-GCM añade su etiqueta de autenticación en disco.

Locks compartidos y exclusivos, más la versión. De David se conservó el lease. De Nicolás se tomaron los dos modos: `X` excluye a otro escritor y `S` permite varios lectores. De María, y también del control de generación de Nicolás, se tomó la versión: `close` publica solo si la versión base sigue vigente. Si no, responde `VERSION_CONFLICT`. El lease dura 30 segundos y, en el anexado, el cliente lo renueva cada 10.

Sin edición parcial. Se adoptó la semántica de David y de Nicolás. `write` crea, reemplaza o anexa. La edición de un rango dentro de un bloque, con copy-on-write, queda como trabajo futuro a partir del diseño de María. Publicar el archivo entero en `close` tiene una comprobación directa: o el otro cliente ve la versión anterior, o ve la nueva.

Heartbeats y reporte de bloques. El heartbeat cada 3 segundos está en los tres diseños. El inventario periódico está explícito en el de María, y Nicolás lo usaba sobre todo cuando un nodo volvía. La implementación hace las dos cosas: a los 10 segundos sin heartbeat el nodo queda sospechoso, a los 30 queda caído, y al arrancar y cada hora envía la lista de bloques para reconciliarla con el estado confirmado.

Docker en una sola VM. El diseño de Nicolás pedía un proceso por máquina virtual, para que apagar la VM fuera el fallo del nodo. El de María ya desplegaba con Docker. En AWS Academy la VM disponible tenía 2 vCPU, 908 MB de RAM y 8 GB de disco, así que los nueve procesos corren como contenedores de una misma instancia, en la red `dfsha`. Parar un contenedor sigue siendo el fallo de ese nodo, y la captura de red

muestra tráfico real entre ellos. No es un fallo de máquinas separadas. Esa diferencia queda declarada en las limitaciones.

Cifrado en el nodo, no de extremo a extremo. El diseño de Nicolás cifraba en el cliente: el nodo guardaba y copiaba el cifrado, y solo una mayoría de servidores de metadatos podía entregar la clave. La implementación cifra en el nodo de datos, con AES-256-GCM y una clave derivada por bloque y versión desde la clave maestra del nodo. Se cambió porque así el cifrado en reposo y la detección de alteraciones quedan en el mismo lugar que almacena el bloque, sin un KMS. El nodo sí ve el claro para servirlo. Junto con eso, el canal Raft usa la contraseña de PySyncObj y la red privada, no mTLS, y el registro de usuarios quedó abierto. En el diseño de Nicolás la cuenta la creaba el operador y todos los canales, incluido el de los controladores, iban con TLS. Esas tres diferencias están en la sección 11.

4. Registro de decisiones arquitectónicas del equipo

4.1 Decisión sobre la arquitectura asignada

Se adopta la arquitectura cliente-servidor con servicios distribuidos entre servidores (S2S).

4.2 Topología y cantidad de nodos

La topología propuesta estará compuesta por:

- Tres servidores de metadatos.
- Cinco nodos de datos.
- Uno o varios clientes.
- Una red privada para la comunicación entre los componentes.

Los tres servidores de metadatos conformarán un clúster Raft. Uno de ellos actuará como líder y los otros dos como seguidores. El líder recibirá las operaciones que modifican el estado de los metadatos y coordinará su replicación.

Los cinco nodos de datos almacenarán los bloques y sus respectivas réplicas. La distribución se realizará procurando que las copias de un mismo bloque permanezcan en nodos diferentes.

4.3 Responsabilidades de los nodos

Cliente:

- Autenticar al usuario.
- Solicitar operaciones de administración de archivos.

- Dividir los archivos en bloques.
- Consultar la ubicación de los bloques.
- Transferir bloques mediante streaming.
- Reconstruir los archivos descargados.
- Verificar la integridad del contenido.
- Administrar las sesiones de lectura y escritura.

Servidores de metadatos:

- Administrar el espacio de nombres.
- Registrar usuarios y permisos.
- Mantener la relación entre archivos, versiones y bloques.
- Coordinar la asignación de nodos de datos.
- Administrar leases y bloqueos.
- Registrar las confirmaciones de escritura.
- Coordinar la recuperación y re-replicación.
- Mantener la consistencia mediante Raft.

Nodos de datos:

- Almacenar bloques.
- Recibir y enviar bloques.
- Mantener réplicas.
- Verificar hashes.
- Participar en la replicación en cadena.
- Enviar heartbeats a los servidores de metadatos.
- Reportar periódicamente los bloques almacenados.
- Ejecutar órdenes de copia y eliminación.

4.4 Distribución y tamaño de bloques

Se establece un tamaño de bloque de 8 MiB configurable. Esta decisión se adopta porque permite dividir archivos grandes en unidades manejables y facilita su transferencia mediante streaming, sin requerir que el contenido completo permanezca en memoria durante la operación.

Cada bloque contará con un identificador asociado al archivo de origen, su posición dentro del archivo y la versión correspondiente. También se conservarán atributos como el tamaño y el hash, necesarios para su identificación y verificación.

4.5 Replicación y ubicación de réplicas

Se adopta un factor de replicación de tres copias por bloque, cada una en un nodo de datos diferente, para que la pérdida de un único nodo no implique la pérdida de la información.

Una escritura se confirma cuando al menos dos de las tres réplicas han almacenado correctamente el bloque y enviado su confirmación. La transferencia se hace en cadena (pipeline): el cliente envía el bloque al primer nodo, este lo reenvía al segundo y el segundo al tercero. El procedimiento detallado se describe en la sección 6.8.

Si un nodo deja de estar disponible, el sistema usa las réplicas restantes y programa la creación de una nueva copia para recuperar el factor de replicación.

4.6 Gestión y almacenamiento de metadatos

Los metadatos se administrarán mediante un clúster de tres servidores coordinados con Raft.

El estado lógico incluirá:

- Identificadores de usuarios.
- Espacios de nombres y directorios.
- Rutas y nombres de archivos.
- Tamaño lógico de los archivos.
- Identificadores y orden de los bloques.
- Versiones de los archivos.
- Estado de las operaciones de escritura.
- Información de bloqueos y leases.
- Relación lógica entre archivos y bloques.

El estado físico de los nodos de datos podrá reconstruirse mediante reportes periódicos de los bloques almacenados. De esta manera, los metadatos lógicos se mantienen coordinados mediante Raft, mientras que la ubicación física puede actualizarse cuando los nodos se recuperan o se incorporan nuevos servidores.

4.7 Protocolos de comunicación

El sistema utiliza gRPC sobre HTTP/2 con Protocol Buffers para definir servicios y mensajes estructurados. La distribución exacta de los canales y la interacción entre los distintos componentes se describe en detalle en la sección 6.1.

4.8 Consistencia y sincronización

La consistencia de los metadatos se mantiene mediante un clúster de tres servidores coordinados con Raft. Las operaciones que modifican el estado lógico del sistema deben ser confirmadas por una mayoría (al menos dos servidores) antes de considerarse definitivas. Para las escrituras de datos se utiliza el quorum, garantizando que los clientes no accedan a versiones incompletas.

La publicación de un archivo se realizará después de completar la transferencia de sus bloques y verificar que se cumpla el criterio de replicación. De esta manera, los clientes no accederán a archivos incompletos o a versiones que no hayan sido confirmadas.

La arquitectura prioriza la consistencia sobre la disponibilidad de escritura cuando no es posible alcanzar los quorums necesarios.

4.9 Concurrencia, locks y control de versiones

Para evitar conflictos entre operaciones simultáneas se usan bloqueos basados en leases renovables, con una duración inicial de 30 segundos. Un cliente que necesita modificar un archivo solicita el bloqueo al servidor de metadatos, que asigna un lease asociado a la sesión y a la operación. Si el cliente deja de renovarlo, el bloqueo expira y queda disponible para otro cliente.

Además, cada archivo tiene una versión asociada. Antes de publicar una nueva versión, el servidor verifica que la versión base no haya cambiado de manera incompatible, de modo que ninguna escritura sobrescribe en silencio cambios confirmados por otro cliente. El algoritmo se describe en la sección 6.9 y su implementación en la 7.5.4.

4.10 Detección de fallos y recuperación

Los nodos de datos envían heartbeats cada tres segundos a los servidores de metadatos, que clasifican cada nodo como ALIVE, SUSPECT o DEAD según el tiempo transcurrido desde el último (ver tabla en la sección 7.8). Un nodo que deja de estar disponible no recibe nuevas asignaciones de bloques. Mientras tanto, las lecturas se atienden con las réplicas restantes y los servidores de metadatos coordinan la creación de nuevas copias hasta recuperar el factor de replicación (sección 5.10). Al reincorporarse, el nodo envía un reporte de sus bloques para contrastarlo con el estado lógico registrado.

4.11 Seguridad

La seguridad se organizará en tres aspectos principales:

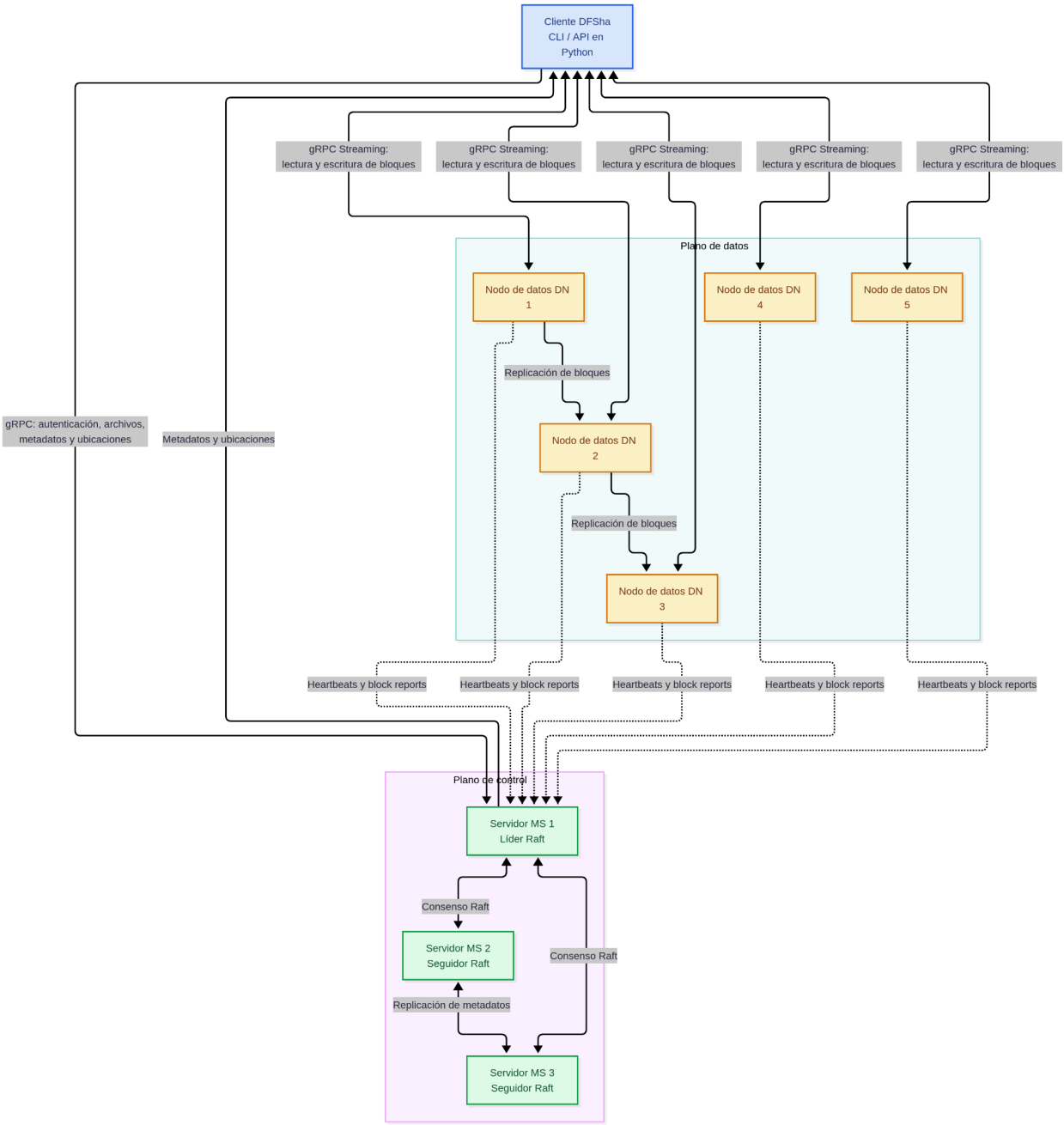
Autenticación: los usuarios deberán autenticarse antes de realizar operaciones sobre el sistema. Se usaron tokens de sesión JWT, para identificar al usuario y validar sus solicitudes.

Autorización: cada solicitud deberá verificar que el usuario tenga permisos sobre el archivo o directorio solicitado. El espacio de almacenamiento será privado por usuario.

Protección de comunicaciones y almacenamiento: las comunicaciones gRPC se protegen con TLS con autenticación mutua, y los bloques de datos se cifran en reposo. Las limitaciones de este modelo (Raft sin mTLS, sin KMS y sin cifrado extremo a extremo) se describen en las secciones 7.9 y 11.1.

5. Arquitectura final de DFSha

5.1 Diagrama de componentes



5.2 Diagrama de despliegue



5.3 Plano de control y plano de datos

El plano de control administra la información necesaria para coordinar el sistema. Incluye los servidores de metadatos, el consenso Raft, la administración de usuarios, los permisos, los bloqueos y la ubicación lógica de los bloques.

El plano de datos se encarga del almacenamiento y transferencia de los bloques. Incluye los nodos de datos y las comunicaciones de streaming entre clientes y servidores, así como el pipeline de replicación entre nodos.

5.4 Flujo de carga de archivos

Cuando un usuario ejecuta la operación put, el cliente inicia el proceso de carga siguiendo estos pasos:

- El cliente autentica al usuario y solicita iniciar la carga.
- El servidor de metadatos valida los permisos y registra una operación temporal.
- El cliente divide el archivo en bloques de 8 MiB.
- Para cada bloque, el cliente solicita al servidor de metadatos los nodos de datos asignados.
- El cliente envía el bloque al primer nodo mediante gRPC streaming.
- El primer nodo almacena el bloque y lo reenvía al segundo nodo.
- El segundo nodo almacena el bloque y lo reenvía al tercero.
- Cada nodo genera su confirmación de almacenamiento.

- Cuando el cliente recibe al menos dos confirmaciones válidas (quorum de escritura), el bloque se considera replicado y se registra en la operación temporal.
- El cliente repite el proceso para los demás bloques.
- Una vez transferidos todos los bloques, solicita finalizar la operación.
- Los servidores de metadatos confirman la nueva versión y publican el archivo.

Si la operación falla antes de completar la confirmación, el archivo permanecerá en estado temporal y no será visible como una versión definitiva.

5.5 Flujo de lectura y recuperación de archivos

Cuando un usuario solicita recuperar un archivo mediante get, el cliente realiza el siguiente procedimiento:

1. Solicita a los servidores de metadatos la información del archivo.
2. Los servidores validan los permisos del usuario.
3. Se obtiene el tamaño del archivo, la versión confirmada y la lista ordenada de bloques.
4. Se consulta la ubicación de las réplicas disponibles para cada bloque.
5. El cliente solicita los bloques a los nodos de datos mediante streaming.
6. Cada bloque recibido se verifica mediante SHA-256.
7. Si un bloque no puede recuperarse o su hash no coincide, se intenta obtener otra réplica.
8. El cliente ordena los bloques de acuerdo con sus identificadores y reconstruye el archivo.
9. Finalmente, se verifica que el archivo reconstruido coincida con el tamaño y el hash esperado, cuando se mantenga un hash global.

Este proceso permite recuperar archivos incluso cuando alguna de las réplicas no está disponible.

5.6 Flujo de escritura y publicación de cambios

Las operaciones write() se realizarán sobre una nueva versión temporal del archivo.

Primero, el cliente solicita autorización y, cuando corresponda, un bloqueo exclusivo. Los servidores de metadatos registran la versión base y asignan los recursos necesarios para la escritura.

El cliente envía los bloques modificados o nuevos a los nodos de datos. Cada bloque se almacena temporalmente y se replica. El sistema verifica la integridad y espera el quorum mínimo de dos confirmaciones.

Cuando todos los bloques necesarios se encuentran confirmados, el cliente ejecuta close() y solicita publicar la nueva versión. El servidor de metadatos valida que la versión base siga siendo válida y registra la nueva versión mediante el mecanismo de consenso.

La publicación es atómica desde el punto de vista lógico: los nuevos lectores acceden a la versión anterior o a la nueva versión confirmada, pero no a una combinación de ambas.

5.7 Flujo de borrado y gestión de directorios

Las operaciones de gestión de directorios y archivos se procesarán mediante los servidores de metadatos.

Para crear un directorio, el cliente enviará una solicitud mkdir. El servidor validará la ruta, los permisos y la existencia del directorio padre antes de registrar el cambio.

Para listar el contenido se utilizará ls, que consultará el espacio de nombres y devolverá las entradas autorizadas.

La operación rmdir permitirá eliminar directorios vacíos. Si el directorio contiene archivos u otros directorios, la operación será rechazada.

La operación rm eliminará lógicamente el archivo y actualizará los metadatos. La eliminación física de los bloques podrá realizarse posteriormente mediante órdenes a los nodos de datos, evitando que una interrupción durante el borrado deje referencias lógicas inconsistentes.

5.8 Distribución de bloques y localización de datos

Cada archivo se identificará mediante un identificador lógico. Los bloques se identificarán utilizando el identificador del archivo, el número de bloque y la versión correspondiente.

Los servidores de metadatos mantendrán la relación entre los archivos y sus bloques, así como las ubicaciones conocidas de las réplicas.

La asignación de nodos deberá considerar únicamente aquellos que se encuentren disponibles y evitar que dos réplicas del mismo bloque terminen en el mismo nodo.

El cliente no necesita conocer de antemano las direcciones de los nodos de datos. Obtendrá las ubicaciones mediante las respuestas de los servidores de metadatos, cumpliendo el requisito de transparencia de localización.

5.9 Replicación y recuperación de bloques

La replicación usa un factor de tres, con el pipeline descrito en la sección 6.8. Cuando un nodo deja de estar disponible, los servidores de metadatos identifican los bloques afectados y verifican si quedan réplicas válidas para seguir atendiendo lecturas. Si el número de réplicas es inferior al factor establecido, programan nuevas copias a partir de las réplicas válidas.

Cuando un nodo se reincorpora, envía un reporte de sus bloques. Los servidores de metadatos lo comparan con el estado lógico y determinan si cada bloque debe conservarse, actualizarse o eliminarse. Los umbrales de detección se detallan en la sección 7.8.

6. APIs, protocolos y algoritmos

6.1 Protocolos de comunicación

La comunicación se organiza en los siguientes canales especializados:

- Entre Cliente y Metadatos: autenticación, gestión de rutas, asignación de bloques y apertura/cierre de archivos.
- Entre Cliente y Nodos de datos: transferencia de bloques mediante streaming (lectura y escritura).
- Entre Nodo y Nodo: pipeline de replicación en cadena.
- Entre Metadatos y Metadatos: coordinación del consenso Raft.
- Entre Nodo y Metadatos: envío de heartbeats, reportes de inventario de bloques y recepción de órdenes administrativas.
-

6.2 API del cliente

La interfaz del cliente expondrá operaciones de alto nivel para que el usuario pueda administrar sus archivos sin interactuar directamente con los nodos de datos.

Operación	Descripción
register()	Registrar un usuario

login()	Iniciar sesión
ls(path)	Listar un directorio
mkdir(path)	Crear un directorio
rmdir(path)	Eliminar un directorio vacío
rm(path)	Eliminar un archivo
put(local, remote)	Cargar un archivo
get(remote, local)	Descargar un archivo
open(path, mode)	Abrir un archivo
read(handle, size)	Leer contenido
write(handle, data)	Escribir contenido
close(handle)	Cerrar y confirmar una operación
lock(path)	Solicitar bloqueo exclusivo
unlock(path)	Liberar bloqueo

6.3 API de los nodos de control

Los servidores de metadatos expondrán operaciones equivalentes a:

Método	Responsabilidad
Authenticate	Validar credenciales y emitir un token
CreateFile	Crear una entrada lógica de archivo

OpenFile	Abrir un archivo y obtener su versión
AllocateBlock	Asignar nodos para almacenar un bloque
GetBlockLocations	Consultar las ubicaciones de las réplicas
CommitBlock	Registrar la confirmación de un bloque
CommitFile	Publicar una nueva versión del archivo
DeleteFile	Registrar el borrado lógico
AcquireLock	Solicitar un bloqueo
ReleaseLock	Liberar un bloqueo
Heartbeat	Recibir el estado de los nodos
BlockReport	Recibir el inventario de bloques de un nodo

6.4 API de los nodos de datos

Los nodos de datos ofrecerán operaciones para recibir, consultar, replicar y eliminar bloques.

Método	**Responsabilidad**
WriteBlock	Recibir y almacenar un bloque
ReadBlock	Enviar un bloque al cliente
ReplicateBlock	Enviar una copia a otro nodo
DeleteBlock	Eliminar un bloque

GetBlockStatus	Consultar el estado de un bloque
SendHeartbeat	Reportar disponibilidad
SendBlockReport	Reportar los bloques almacenados

6.5 Formato de mensajes y respuestas

Los mensajes deberán contener la información necesaria para identificar la operación y validar los datos transferidos.

Un mensaje de escritura de bloque podrá incluir:

- Identificador del archivo.
- Identificador de bloque.
- Número de bloque.
- Versión del archivo.
- Tamaño del contenido.
- Hash SHA-256.
- Identificador de sesión.
- Token de autorización.
- Datos del bloque o flujo de bytes.

Las respuestas deberán indicar si la operación fue exitosa, si requiere reintento o si se presentó un error.

6.6 Manejo de errores

El sistema deberá manejar los siguientes escenarios:

Error	Situación	Comportamiento esperado
UNAUTHENTICATED	Token inválido o ausente	Solicitar autenticación
PERMISSION_DENIED	Usuario sin permisos	Rechazar la operación

NOT_FOUND	Archivo o bloque inexistente	Informar que no existe
ALREADY_EXISTS	Archivo o directorio duplicado	Rechazar la creación
NOT_LEADER	Solicitud enviada a un seguidor	Redirigir al líder
NO_QUORUM	No existe mayoría Raft	No confirmar la modificación
INSUFFICIENT_REPLICAS	No se alcanza el quorum de datos	No publicar la escritura
VERSION_CONFLICT	La versión base cambió	Rechazar y solicitar reintento
BLOCK_LOCKED	Otro cliente tiene el bloqueo	Esperar o reintentar
DATA_LOSS	Bloque ilegible o corrupto	Intentar recuperar otra réplica
UNAVAILABLE	Nodo no disponible	Reintentar o usar otra réplica

6.7 Algoritmos de particionamiento y distribución

Para asignar las réplicas se utilizarán los nodos de datos que estén marcados como disponibles. El algoritmo deberá seleccionar tres nodos distintos para cada bloque y distribuir las asignaciones entre el conjunto de nodos.

La selección podrá realizarse mediante una función determinista basada en el identificador del archivo y el índice del bloque, aplicada sobre la lista de nodos disponibles. El objetivo es distribuir los bloques de manera equilibrada y evitar que todas las réplicas de un bloque se encuentren en una misma máquina.

El procedimiento general será:

1. Obtener la lista de nodos de datos disponibles.
2. Recibir el identificador del archivo y el índice del bloque.
3. Calcular el orden de selección de los nodos utilizando la función de asignación.
4. Seleccionar tres nodos diferentes para almacenar las réplicas.

5. Registrar la asignación en los servidores de metadatos.
6. Devolver al cliente la información necesaria para iniciar la transferencia.

6.8 Algoritmos de replicación y consistencia

La replicación se realiza mediante pipeline. El procedimiento de escritura de cada bloque es:

1. El cliente envía el bloque al primer nodo asignado.
2. El primer nodo recibe el contenido, verifica su integridad (SHA-256), lo almacena y lo reenvía al segundo.
3. El segundo nodo repite el proceso y reenvía el bloque al tercero.
4. Cada nodo genera una confirmación según el resultado de su almacenamiento.
5. Cuando se alcanza el quorum mínimo de dos confirmaciones, el bloque se considera confirmado y el servidor de metadatos registra la confirmación.

La sincronización del estado lógico se delega en Raft: las transacciones de metadatos se procesan únicamente cuando el líder logra la confirmación de la mayoría de los servidores (sección 4.8).

6.9 Algoritmos de concurrencia y recuperación

Los bloqueos se implementan como leases con vencimiento, asociados a un cliente y a una operación. El servidor de metadatos controla su vigencia e impide que dos clientes mantengan simultáneamente un bloqueo exclusivo válido sobre el mismo recurso. El procedimiento es:

1. El cliente solicita un bloqueo sobre el archivo.
2. El servidor verifica si existe otro bloqueo exclusivo válido.
3. Si el recurso está disponible, asigna un lease de 30 segundos.
4. El cliente renueva el lease mientras mantiene activa la operación.
5. Si el lease vence, el bloqueo se libera para nuevas solicitudes.
6. Al finalizar, el cliente libera el bloqueo.

El control de versiones detecta modificaciones hechas por otros clientes durante una sesión abierta: antes de publicar, el sistema verifica que la versión base sea la esperada.

Para la recuperación en lectura, el cliente prioriza los nodos con menor latencia reportada y descarta los que devuelven errores de validación SHA-256. Una réplica con errores de integridad deja de usarse como fuente y el bloque se solicita a otra ubicación.

7. Implementación

7.1. Lenguajes, frameworks y herramientas

Categoría	Tecnología	Función en el sistema
Lenguaje	Python 3.11	Implementación del cliente, servidores de metadatos y nodos de datos.
Comunicación	gRPC y HTTP/2	Intercambio de solicitudes y respuestas entre componentes.
Serialización	Protocol Buffers	Definición de mensajes y contratos de los servicios.
Consenso	PySyncObj (Raft)	Replicación del estado de metadatos entre tres servidores.
Integridad	SHA-256	Verificación de los bloques durante escritura, lectura y mantenimiento.
Autenticación	Argon2id y PyJWT	Protección de contraseñas y gestión de sesiones de usuario.
Cifrado en reposo	AES-256-GCM y HKDF-SHA256	Protección de los bloques almacenados en los nodos de datos.
Seguridad de transporte	TLS y mTLS	Protección de las comunicaciones gRPC y autenticación de los nodos.
Contenedores	Docker y Docker Compose	Definición y orquestación de los servicios del clúster.
Pruebas	pytest	Ejecución de pruebas funcionales, de concurrencia, fallos, replicación y rendimiento.

Automatización	Bash	Generación de certificados y stubs, arranque, parada y limpieza del clúster.
Infraestructura prevista	AWS Academy, EC2, VPC y EBS	Entorno previsto para el despliegue distribuido.
Control de versiones	Git y GitHub	Gestión del código fuente y colaboración del equipo.

7.2. Organización del código fuente

None

dfsha/

├── client/

│ ├── cli.py

│ ├── dfs_client.py

│ ├── metadata_client.py

│ ├── data_node_client.py

│ ├── main.py

│ └── Dockerfile

├── metadata-server/

│ ├── metadata_service.py

│ ├── user_manager.py

│ ├── namespace_manager.py

│ ├── file_manager.py

│ ├── block_manager.py

│ ├── replication_manager.py

│ ├── storage/

│ │ └── metadata_repository.py

```
│   ├── main.py
│   └── Dockerfile
├── data-node/
│   ├── data_service.py
│   ├── block_storage.py
│   ├── integrity.py
│   ├── replication.py
│   ├── heartbeat.py
│   ├── storage/
│   ├── main.py
│   └── Dockerfile
├── proto/
│   ├── auth.proto
│   ├── metadata.proto
│   ├── data_node.proto
│   └── generated/
├── shared/
│   ├── config.py
│   ├── constants.py
│   ├── exceptions.py
│   ├── security.py
│   └── utils.py
│
├── scripts/
│   ├── generate_proto.sh
```

```
│   ├── generate_certs.sh
│   ├── start_cluster.sh
│   ├── stop_cluster.sh
│   └── clean_storage.sh
├── tests/
│   ├── functional/
│   ├── concurrency/
│   ├── failures/
│   ├── replication/
│   ├── performance/
│   ├── conftest.py
│   └── helpers.py
├── docs/
├── docker-compose.yml
├── requirements.txt
├── requirements-dev.txt
└── README.md
```

La carpeta client/ contiene la interfaz de línea de comandos, la biblioteca de alto nivel y los clientes gRPC utilizados para comunicarse con los servidores de metadatos y los nodos de datos.

La carpeta metadata-server/ agrupa los servicios responsables de administrar usuarios, directorios, archivos, sesiones de escritura, bloqueos y asignación de bloques. También contiene el repositorio que mantiene el estado lógico replicado mediante Raft.

La carpeta data-node/ contiene los servicios encargados de recibir, almacenar, leer, replicar y eliminar bloques. Incluye los módulos de integridad, almacenamiento cifrado, replicación y envío de heartbeats.

shared/ centraliza las constantes, configuración, excepciones, utilidades y funciones de seguridad que utilizan los diferentes componentes. Esto evita duplicar implementaciones y permite mantener parámetros comunes, como el tamaño de bloque, los tiempos de expiración y el factor de replicación.

Los archivos .proto son la fuente de los contratos de comunicación. Los stubs generados se producen mediante scripts/generate_proto.sh y se guardan en proto/generated/. Esta carpeta no contiene los contratos originales, sino el código Python generado para utilizarlos desde los servicios.

Finalmente, tests/ agrupa las pruebas según las propiedades del sistema que verifican, mientras que scripts/ contiene los procedimientos de automatización necesarios para preparar y administrar el clúster.

7.3. Despliegue y configuración de los componentes

Cada servidor de metadatos dispone de un volumen independiente para conservar su estado de Raft, incluyendo el journal y los archivos de volcado. De manera similar, cada nodo de datos utiliza un volumen independiente para almacenar sus bloques cifrados. El cliente dispone de un volumen para conservar su información de sesión.

Los servicios se conectan mediante una red privada definida en docker-compose.yml. Esta configuración permite que los componentes se comuniquen utilizando sus nombres de servicio, sin necesidad de exponer los nodos de almacenamiento directamente a redes externas.

Los principales puertos utilizados son:

- 7000: comunicación interna de Raft entre servidores de metadatos.
- 7100: servicio gRPC de los servidores de metadatos.
- 7001: servicio gRPC de los nodos de datos.

En el modo local de desarrollo se utilizan puertos diferentes para distinguir los procesos de cada componente. Los scripts de inicio permiten levantar tres servidores de metadatos y cinco nodos de datos sobre una misma máquina.

El procedimiento de preparación del clúster contempla la generación de certificados y secretos, la construcción de las imágenes y el inicio de los servicios. Para ello se dispone de los scripts generate_certs.sh y start_cluster.sh. Los procedimientos de parada y limpieza se encuentran en stop_cluster.sh y clean_storage.sh.

7.4. Implementación del cliente

El cliente constituye el punto de entrada al sistema. Se implementa mediante dos módulos principales: dfs_client.py, que proporciona la biblioteca de operaciones, y cli.py, que expone dichas operaciones mediante una interfaz de línea de comandos.

La biblioteca DFSClient integra dos clientes de comunicación:

- MetadataClient: permite consultar y modificar los metadatos, administrar sesiones y obtener la ubicación de los bloques.
- DataNodeClient: permite transferir bloques directamente entre el cliente y los nodos de datos.

Esta separación mantiene la distinción entre el plano de control y el plano de datos. El cliente no necesita conocer previamente las direcciones de los nodos de almacenamiento, ya que estas se obtienen a partir de las respuestas del servidor de metadatos.

7.4.1. Gestión de sesiones y autenticación

El cliente utiliza una sesión local para conservar la información necesaria entre ejecuciones. Esta sesión incluye el token JWT, el identificador del usuario, el directorio de trabajo actual y la identidad del cliente.

La información se almacena en un archivo local con permisos 0600, restringiendo su lectura y modificación al usuario propietario del archivo.

Cuando un usuario ejecuta register, el cliente solicita el registro de sus credenciales. Posteriormente, mediante login, obtiene un JWT que utiliza en las solicitudes autenticadas dirigidas a los servidores de metadatos.

El cliente también conserva un identificador propio (cid), utilizado como identidad del titular de los bloqueos. Esto permite distinguir clientes diferentes incluso cuando pertenecen al mismo usuario.

7.4.2. Operaciones de gestión de archivos y directorios

Cada ruta relativa se transforma en una ruta absoluta tomando como referencia el directorio de trabajo del usuario. Las operaciones remotas se envían a los servidores de metadatos, que verifican la existencia de los recursos y los permisos correspondientes.

El espacio de nombres de cada usuario se organiza bajo /users/<uid>. Las rutas se validan para impedir que un usuario acceda a recursos ubicados fuera de su propio directorio.

7.4.3. Transferencia de archivos completos

La operación put permite transferir un archivo desde el sistema local hacia DFSha. El cliente lee el archivo de origen en segmentos de hasta 8 MiB y entrega cada segmento al mecanismo de escritura de bloques. El procedimiento continúa hasta completar el archivo y cerrar la sesión de escritura.

La operación get realiza el proceso inverso: consulta los metadatos, obtiene la ubicación de los bloques, descarga su contenido y reconstruye el archivo en la ruta local indicada.

La descarga puede ejecutarse en paralelo mediante un grupo de trabajadores. Sin embargo, los bloques se entregan al archivo de destino respetando su orden original, de modo que la reconstrucción conserve la secuencia de bytes del archivo.

Ambas operaciones registran el tamaño transferido y el tiempo empleado, lo que permite obtener mediciones básicas de rendimiento**.**

7.4.4. Operaciones de archivo abierto

Para RF3 se implementó la abstracción FileHandle, que administra el ciclo de vida de un archivo mediante las operaciones open, read, write y close.

Se contemplan tres modos:

- r: abre un archivo para lectura.
- w: crea un archivo o reemplaza su contenido.
- a: anexa contenido al final de un archivo existente.

En el modo de lectura, el cliente obtiene los metadatos y la versión vigente del archivo al momento de abrirlo. Esto permite mantener una referencia a una versión específica durante la sesión de lectura.

En los modos de escritura, el cliente crea una sesión en el servidor de metadatos. Los datos se acumulan temporalmente en un búfer y se convierten en bloques de 8 MiB cuando alcanzan el tamaño establecido. El último bloque puede tener un tamaño inferior.

La implementación utiliza un mecanismo de escritura por nuevas versiones: los bloques nuevos se almacenan y confirman sin modificar inmediatamente la versión visible del archivo. La publicación ocurre al ejecutar close, una vez que todos los bloques han sido confirmados.

La modificación parcial arbitraria de un bloque ya almacenado no está implementada. Las operaciones disponibles son crear, reemplazar y anexar.

7.5. Implementación de los servidores de metadatos

Los servidores de metadatos administran el estado lógico de DFSha. La implementación utiliza tres instancias de MetadataRepository, coordinadas mediante PySyncObj y el algoritmo Raft.

Cada servidor mantiene una copia del estado replicado, que comprende:

- Usuarios registrados y sus hashes de contraseña.
- Directorios y archivos del espacio de nombres.
- Identificadores y metadatos de los bloques.
- Ubicación de las réplicas de cada bloque.
- Sesiones de escritura pendientes.
- Versiones de los archivos.
- Bloqueos activos.
- Registros de eliminación pendientes de recolección.

Las modificaciones del estado pasan por el método do(op, args), que constituye la entrada de escritura replicada. Raft aplica las operaciones en el mismo orden entre los servidores que participan en el consenso. Las operaciones que modifican el estado se ejecutan de forma síncrona y requieren una mayoría.

7.5.1. Elección del líder y disponibilidad

En cada momento, uno de los tres servidores actúa como líder de Raft y es responsable de atender las solicitudes de modificación de metadatos.

Los servidores seguidores no procesan directamente las operaciones de metadatos. Cuando reciben una solicitud, responden con NOT_LEADER e incluyen la dirección del líder cuando esta se encuentra disponible. El cliente utiliza esa información para redirigir la solicitud.

Si no se puede obtener una mayoría de Raft, las operaciones de escritura de metadatos fallan con NO_QUORUM. Esto evita confirmar modificaciones que no puedan ser replicadas de acuerdo con el protocolo de consenso.

La configuración de PySyncObj utiliza un tiempo mínimo de elección de un segundo y uno máximo de dos segundos. El estado persistente de cada servidor se conserva mediante un archivo de volcado (raft.dump) y un journal (raft.journal).

7.5.2. Administración de usuarios y permisos

El módulo user_manager.py implementa el registro, la autenticación y la autorización de los usuarios.

Durante el registro, el sistema valida el identificador y la longitud mínima de la contraseña. Durante la autenticación se verifica la contraseña y se emite un JWT con

una duración predeterminada de 900 segundos (15 minutos). El tratamiento de contraseñas y la firma del token se detallan en la sección 7.9.

La autorización de rutas comprueba que cada operación se mantenga dentro del directorio /users/<uid>, de modo que cada usuario dispone de un espacio de nombres independiente.

7.5.3. Administración del espacio de nombres

El módulo namespace_manager.py administra la creación, consulta y eliminación de directorios y archivos.

La creación de directorios verifica que el directorio padre exista y que el nombre de la ruta no esté ocupado. La operación rmdir solamente permite eliminar directorios vacíos, mientras que rm se utiliza para eliminar archivos.

Cuando se elimina un archivo, este deja de aparecer en el espacio de nombres y sus bloques quedan registrados en una lista de eliminación pendiente. El borrado físico se realiza posteriormente, una vez transcurrido el periodo de gracia configurado.

7.5.4. Administración de archivos, versiones y bloqueos

El módulo file_manager.py controla las sesiones de escritura, la confirmación de bloques y la publicación de nuevas versiones.

Cuando un cliente ejecuta open en modo de escritura, el servidor crea un identificador de sesión y registra la versión base del archivo. Los bloques nuevos se asocian a esa sesión y no forman parte todavía de la versión visible.

Cada bloque se registra con commit_block, que verifica que la sesión exista, que la versión base siga vigente y que el bloque tenga al menos dos confirmaciones de almacenamiento. Después, commit_file comprueba que todos los bloques estén presentes y que el tamaño total coincida con el declarado, publica la nueva versión e incrementa su número. Si la versión vigente cambió desde que comenzó la sesión (expected_version), la operación falla con VERSION_CONFLICT.

Los bloqueos se administran con leases (política general en las secciones 4.9 y 6.9). Se admiten modos compartido (S) y exclusivo (X), tanto sobre un archivo completo como sobre un bloque específico. En el modo de anexado, el cliente solicita un bloqueo exclusivo del archivo y lo renueva cada 10 segundos mientras mantiene abierta la sesión.

7.5.5. Asignación y localización de bloques

El módulo block_manager.py administra el registro de nodos de datos y la asignación de bloques.

Cuando el cliente solicita almacenar un bloque, el servidor consulta los nodos disponibles y selecciona hasta tres nodos diferentes. La selección considera la cantidad de asignaciones realizadas previamente y el espacio libre informado por cada nodo. De esta manera, se intenta distribuir la carga y evitar concentrar las nuevas asignaciones en un único servidor.

La lista de nodos seleccionados se devuelve al cliente como una cadena de replicación o pipeline. La posición inicial de esta cadena rota según el índice del bloque, evitando que todas las transferencias comiencen siempre en el mismo nodo.

Para las lecturas, el servidor consulta las réplicas registradas y devuelve las ubicaciones de los bloques requeridos. Las réplicas consideradas disponibles se mezclan para distribuir las solicitudes de lectura entre los nodos.

7.6. Implementación de los nodos de datos

Los cinco nodos de datos son responsables del almacenamiento físico de los bloques y de las operaciones de transferencia. Cada nodo ejecuta un servicio gRPC definido en data_service.py y utiliza block_storage.py para administrar los archivos almacenados localmente.

El almacenamiento se organiza mediante archivos individuales, identificados por el identificador del bloque. Cada bloque se conserva cifrado y dispone de la información necesaria para recuperar su versión y contenido.

7.6.1. Escritura y almacenamiento físico

Cuando un nodo recibe una solicitud WriteBlock, primero verifica el token de capacidad recibido. Este token debe autorizar la escritura del bloque correspondiente y encontrarse vigente.

El nodo recibe los fragmentos enviados por streaming, reconstruye el bloque completo y calcula su hash SHA-256. Si el valor calculado no coincide con el hash esperado, rechaza la operación.

Cuando la verificación es satisfactoria, el nodo cifra el contenido mediante AES-256-GCM y lo almacena en disco. El formato de almacenamiento contiene:

1. La versión del bloque.
2. Un nonce de 12 bytes.

3. El contenido cifrado y su etiqueta de autenticación.

La clave de cifrado se deriva mediante HKDF-SHA256 a partir de una clave maestra y de la identidad y versión del bloque. Esto permite utilizar una clave derivada diferente para cada combinación de bloque y versión.

Para evitar que un bloque incompleto quede visible si ocurre una interrupción durante la escritura, el sistema utiliza un archivo temporal con extensión .tmp. Primero escribe el contenido, ejecuta fsync y posteriormente reemplaza el archivo definitivo mediante una operación de renombrado. Al iniciar, el nodo elimina los archivos temporales huérfanos que hayan quedado de ejecuciones anteriores.

7.6.2. Lectura de bloques

La operación ReadBlock permite recuperar el contenido de un bloque a partir de su identificador.

Antes de entregar los datos, el nodo verifica el token de lectura. Posteriormente, localiza el archivo almacenado, descifra su contenido y lo envía al cliente mediante streaming gRPC, dividiéndolo en fragmentos de 1 MiB.

El cliente vuelve a calcular el SHA-256 del bloque recibido y lo compara con el valor registrado en los metadatos. Si el contenido no coincide o el nodo no responde, se intenta obtener el bloque desde otra réplica.

De esta manera, la lectura no depende de que una única copia del bloque se encuentre disponible.

7.6.3. Replicación entre nodos

La replicación se implementa con dos mecanismos complementarios.

El primero se usa en la escritura inicial: el cliente envía el bloque al primer nodo de la cadena, este lo almacena antes de reenviarlo al siguiente, y cada nodo devuelve las confirmaciones de los nodos que participaron (algoritmo en la sección 6.8).

El segundo se usa en la recuperación de fallos o cuando un bloque tiene menos de tres réplicas: el servidor de metadatos ordena a un nodo que ya posee el bloque copiarlo a otro mediante ReplicateBlock.

En ambos casos se usa el mismo mecanismo de escritura y verificación de integridad, y la nueva réplica se registra en los metadatos solo cuando la operación devuelve una confirmación satisfactoria.

7.6.4. Integridad y mantenimiento

El módulo integrity.py proporciona las funciones de verificación SHA-256 y el proceso periódico de scrubbing.

Durante el scrubbing, el nodo recorre los bloques almacenados e intenta descifrarlos. AES-256-GCM permite detectar alteraciones del contenido mediante su etiqueta de autenticación. Si un bloque no puede recuperarse correctamente, se elimina del almacenamiento local.

El siguiente reporte de bloques permite que el servidor de metadatos detecte que esa réplica ya no existe y, si corresponde, ordene su reconstrucción a partir de otra copia disponible.

7.7. Flujo de ejecución de las operaciones principales

La interacción entre el cliente, los servidores de metadatos y los nodos de datos permite ejecutar las operaciones de almacenamiento y recuperación sin que los servidores de metadatos tengan que transportar el contenido completo de los archivos.

7.7.1. Escritura de un archivo (put)

El flujo general corresponde al de la sección 5.5. En la implementación, put se ejecuta sobre una sesión de escritura (open en modo w) y utiliza las siguientes llamadas:

El cliente abre la sesión de escritura, y el servidor de metadatos valida la ruta y los permisos, y registra la versión base del archivo.

Por cada bloque de hasta 8 MiB, el cliente invoca AllocateBlock. El servidor responde con el identificador del bloque, su versión, la cadena de replicación (hasta tres nodos) y un token de escritura.

El cliente calcula el SHA-256 del bloque y lo envía al primer nodo mediante WriteBlock, en fragmentos de 1 MiB. Cada nodo verifica el token y el hash, almacena el bloque cifrado (sección 7.6.1) y lo reenvía al siguiente nodo de la cadena (sección 7.6.3).

Cuando el cliente recibe al menos dos confirmaciones, invoca CommitBlock para registrar el bloque en la sesión.

Al terminar todos los bloques, el cliente ejecuta CommitFile mediante close, y el servidor publica la nueva versión (sección 7.5.4).

El quorum de escritura es dos aunque se seleccionen tres nodos. Si la sesión se interrumpe antes de CommitFile, sus bloques no forman parte de la versión visible.

7.7.2. Lectura de un archivo (get y read)

El flujo general corresponde al de la sección 5.6. En la implementación, la lectura se apoya en las siguientes llamadas:

1. El cliente abre el archivo en modo r e invoca GetBlockLocations. El servidor devuelve la versión vigente, el tamaño total y, por cada bloque, su identificador, tamaño, hash, desplazamiento y direcciones de las réplicas disponibles.
2. El cliente solicita cada bloque con ReadBlock. El nodo valida el token de lectura, descifra el bloque y lo envía por streaming (sección 7.6.2).
3. El cliente verifica el SHA-256 contra el valor registrado en los metadatos. Si una réplica no responde o el contenido es inválido, prueba con otra.
4. Si fallan todas las réplicas inicialmente conocidas, el cliente consulta de nuevo los metadatos para obtener ubicaciones actualizadas, siempre que la versión del archivo siga coincidiendo con la de la sesión de lectura.

La descarga paralela de bloques descrita en la sección 7.4.3 no altera el orden ni el contenido del archivo reconstruido.

7.7.3. Escritura mediante open, write y close

Este flujo reutiliza el procedimiento de put: en realidad, put es una aplicación de open, write y close sobre un archivo local. La semántica del búfer, la conversión en bloques de 8 MiB y la publicación en close se describen en la sección 7.4.4, y las verificaciones de commit_file (bloques completos, tamaño total y versión base) en la sección 7.5.4.

Lo específico de esta modalidad es lo siguiente:

- Los datos entregados a write se acumulan en un búfer local. Cada vez que este alcanza 8 MiB, el cliente ejecuta el ciclo de AllocateBlock, WriteBlock y CommitBlock descrito en la sección 7.7.1.
- El bloque final, que puede ser menor de 8 MiB, se envía al ejecutar close.
- Si otro cliente publicó una versión nueva durante la sesión, close falla con VERSION_CONFLICT y no se publica nada.

7.7.4. Eliminación de archivos

La eliminación de archivos utiliza un mecanismo de borrado lógico y recolección física diferida.

Cuando se ejecuta rm, el servidor elimina la referencia al archivo del espacio de nombres y registra una lápida (*tombstone*) que contiene los identificadores de los bloques que deben eliminarse.

El gestor de replicación revisa periódicamente estas lápidas. Cuando transcurre el periodo de gracia configurado, envía solicitudes DeleteBlock a los nodos que

almacenan las réplicas y posteriormente elimina los registros correspondientes de los metadatos.

Este procedimiento evita que la eliminación lógica dependa de que todos los nodos de datos se encuentren disponibles en el mismo instante.

7.8. Detección de fallos y recuperación

Reemplace los dos primeros párrafos (antes de la tabla) por:

La detección de fallos se basa en heartbeats que cada nodo de datos envía cada tres segundos a los servidores de metadatos, con su dirección y el espacio libre del almacenamiento local. El líder mantiene en memoria el último heartbeat de cada nodo y determina tres estados:

El servidor de metadatos mantiene un registro en memoria del líder con el último heartbeat recibido por cada nodo. A partir de ese tiempo se determinan tres estados:

Estado	Condición	Comportamiento
ALIVE	Último heartbeat recibido hace menos de 10 segundos.	El nodo se considera disponible para nuevas operaciones.
SUSPECT	Han transcurrido entre 10 y 30 segundos sin recibir un heartbeat.	El nodo deja de considerarse disponible para asignaciones y se espera una recuperación.
DEAD	Han transcurrido más de 30 segundos sin recibir un heartbeat.	El nodo se considera caído y sus réplicas se retiran del registro de ubicaciones disponibles.

Cuando el gestor de replicación detecta que un nodo está caído, revisa los bloques que tenían una réplica en él. Si un bloque dispone de menos de tres copias registradas y existe al menos una réplica disponible, se programa su reconstrucción en otro nodo.

La selección del nodo destino considera el espacio libre y evita utilizar una ubicación que ya esté registrada para ese bloque. Las tareas de re-replicación se ejecutan de forma asíncrona mediante un grupo de trabajadores.

Además de los heartbeats, los nodos envían un reporte completo de bloques al iniciar y posteriormente cada hora. Estos reportes permiten reconciliar el contenido físico de cada nodo con el registro de metadatos. De esta forma se pueden identificar tanto

réplicas perdidas como bloques huérfanos que ya no están asociados a una sesión o archivo conocido.

El sistema también contempla la expiración de sesiones de escritura abandonadas. Si una sesión permanece sin actividad durante 600 segundos, el servidor la expira y registra sus bloques nuevos para su posterior recolección.

La información de los nodos de datos se mantiene en memoria del líder y se reconstruye mediante los heartbeats y reportes después de una elección de líder. Por este motivo, durante un breve periodo posterior a un cambio de líder, una asignación de bloques puede fallar por disponibilidad insuficiente hasta que se recupere el registro de los nodos.

7.9. Seguridad implementada

La seguridad se incorpora en las operaciones de autenticación, autorización, comunicación y almacenamiento.

Autenticación de usuarios. Las contraseñas se procesan con Argon2id antes de almacenarse. Tras validar las credenciales, el servidor genera un JWT firmado con HS256, que incluye el identificador del usuario y su fecha de expiración y se usa para autenticar las solicitudes posteriores.

Autorización de rutas. El servidor de metadatos comprueba que cada usuario opere dentro de su propio espacio de nombres. Las rutas externas a /users/<uid> se rechazan con un error de permisos.

Tokens de capacidad. Para las operaciones sobre los nodos de datos se usan tokens que autorizan una acción sobre un bloque determinado: escritura (w), lectura (r) o administración (x). Se firman con HMAC-SHA256 y contienen una fecha de expiración.

Cifrado en reposo. Los bloques se almacenan con AES-256-GCM. La clave de cada bloque se deriva con HKDF-SHA256 a partir de la clave maestra del nodo, el identificador del bloque y su versión. Esto protege el contenido y permite detectar modificaciones no autorizadas.

Seguridad del transporte. Los canales gRPC usan TLS con autenticación mutua (mTLS), mediante una autoridad certificadora privada y un certificado por nodo, generados con generate_certs.sh. En el modo local de pruebas no se usa TLS.

Alcance y límites. La replicación Raft entre los servidores de metadatos no usa mTLS: se protege con la contraseña de PySyncObj y la red privada de Docker, por lo que la protección mTLS aplica únicamente a los canales gRPC. Los metadatos replicados por Raft no se cifran en reposo. Tampoco hay cifrado extremo a extremo ni un KMS: las claves maestras se entregan a los nodos mediante variables de entorno.

7.10. Pruebas y validación de la implementación

La validación se realiza mediante pytest, con pruebas organizadas por las propiedades funcionales y no funcionales del sistema.

Las pruebas de integración requieren que el clúster se encuentre en ejecución. Para facilitar su ejecución, tests/conftest.py define las fixtures de clientes y tests/helpers.py proporciona funciones auxiliares para registrar usuarios, iniciar sesiones, consultar réplicas y controlar nodos.

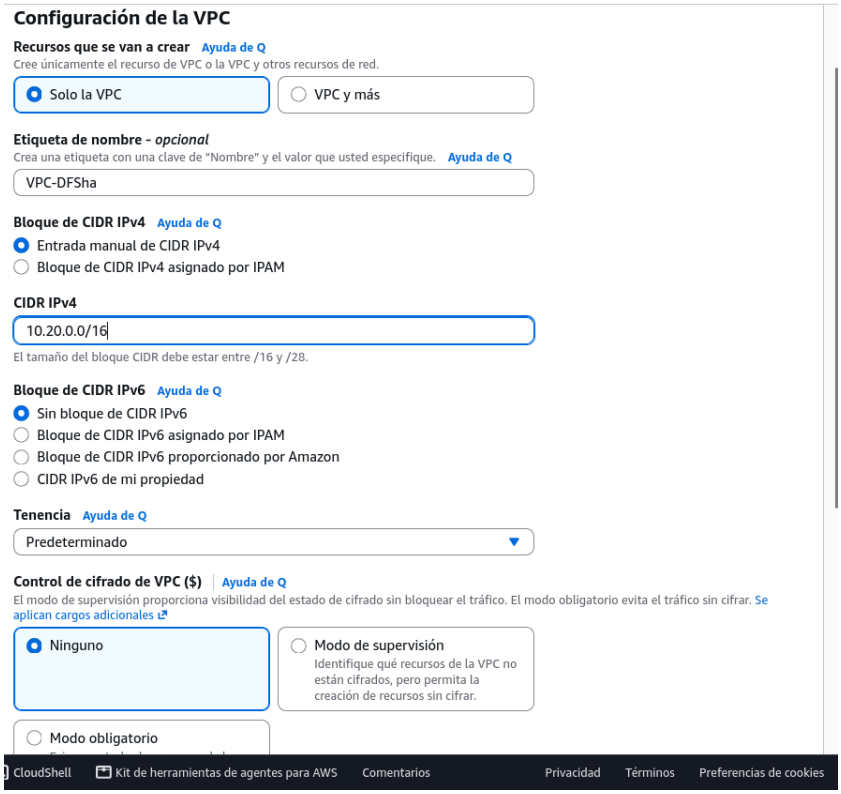
La estructura contempla las siguientes categorías:

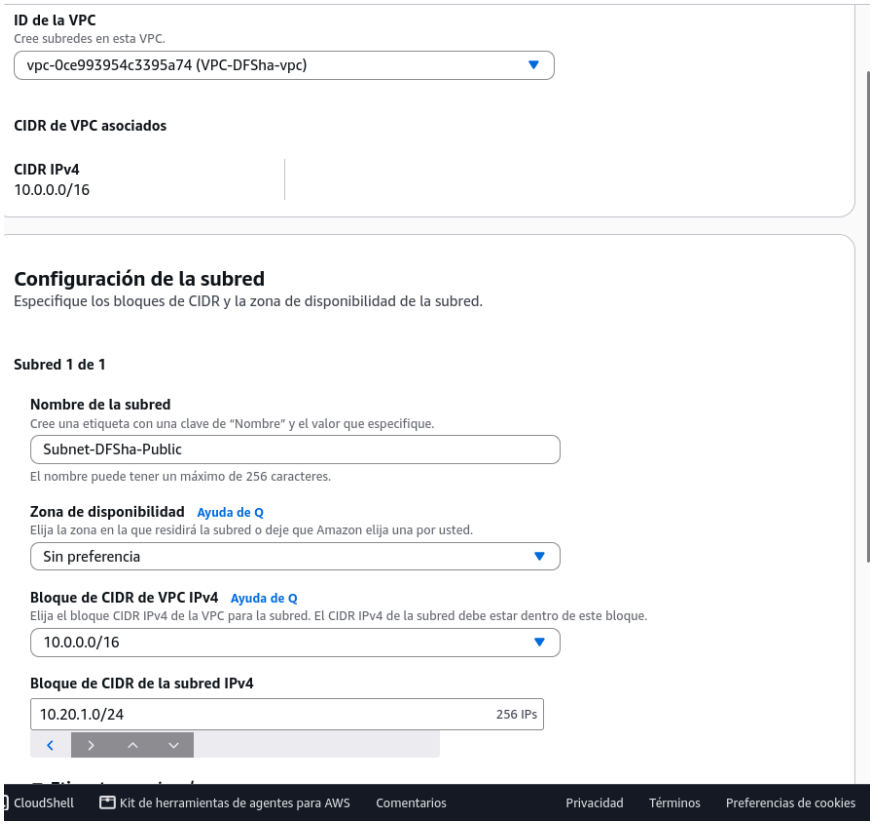
Categoría	Objetivo de validación
Funcionales	Comprobar las operaciones de directorios, archivos, transferencias y operaciones de archivo abierto.
Concurrencia	Verificar el comportamiento de bloqueos, leases y conflictos de versiones entre clientes.
Replicación	Comprobar la existencia de copias, la integridad de los bloques y la recuperación del factor de replicación.
Fallos	Evaluar la continuidad de las operaciones ante la caída de nodos de datos y servidores de metadatos.
Rendimiento	Medir los tiempos y tasas de transferencia de archivos y las operaciones de consulta de metadatos.

8. Despliegue y configuración

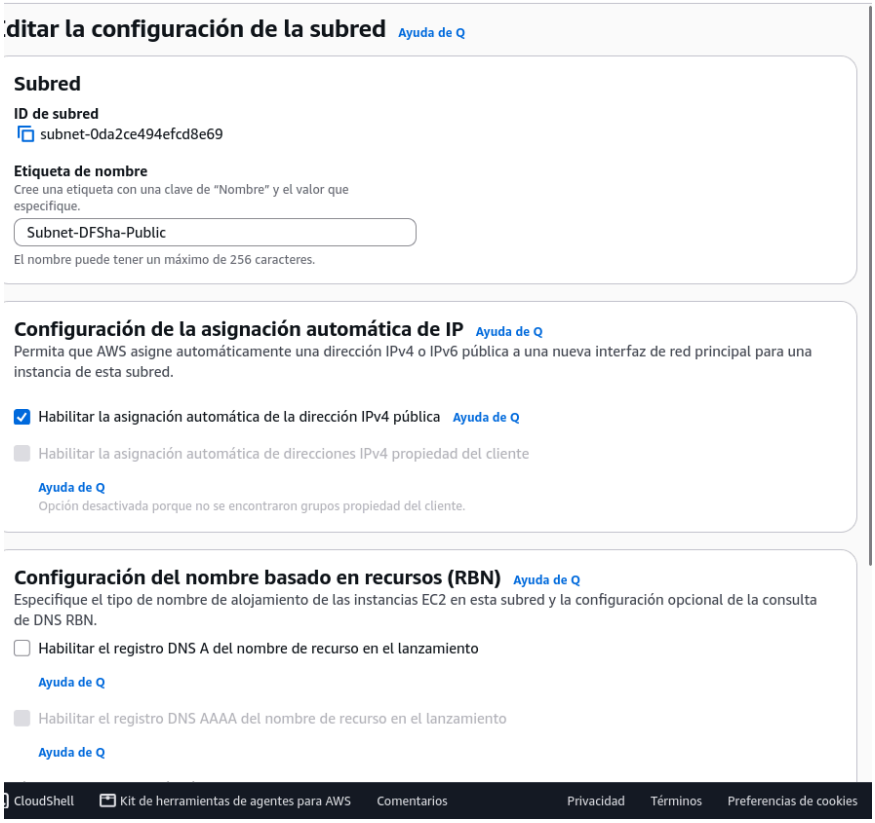
8.1 Infraestructura y configuración de red

Se crea una VPC con un bloque con la ip 10.20.0.0/16 para reservar un rango privado amplio para la VPC, y para la subred 256 direcciones IPv4.

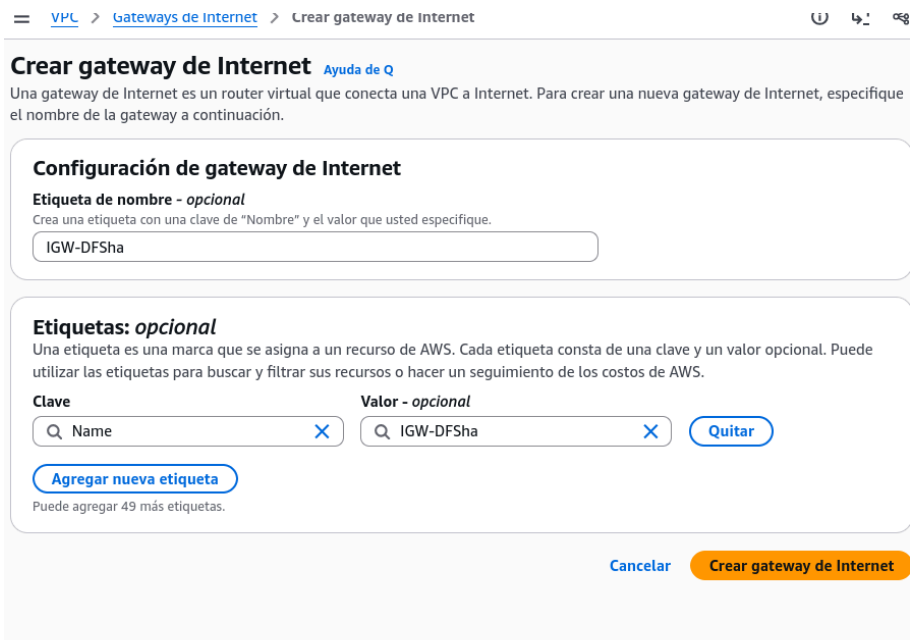


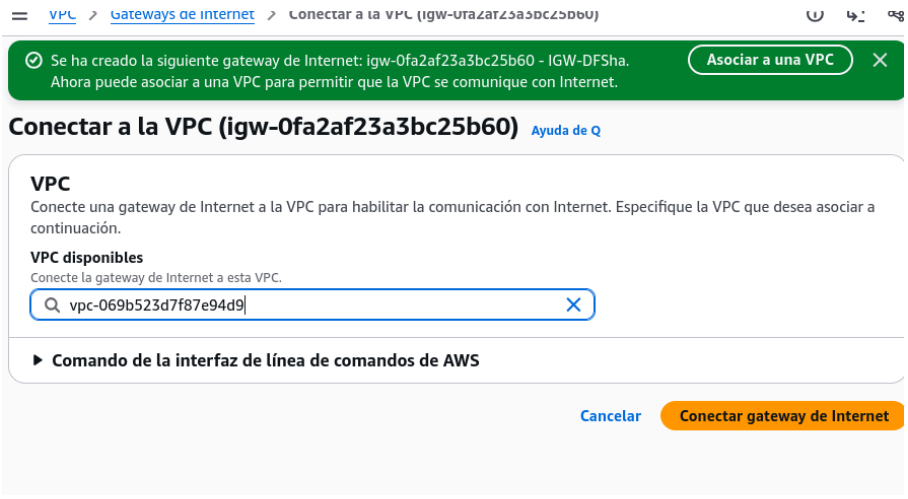


Se le habilita a la subred la asignación automática de ip’s publicas.

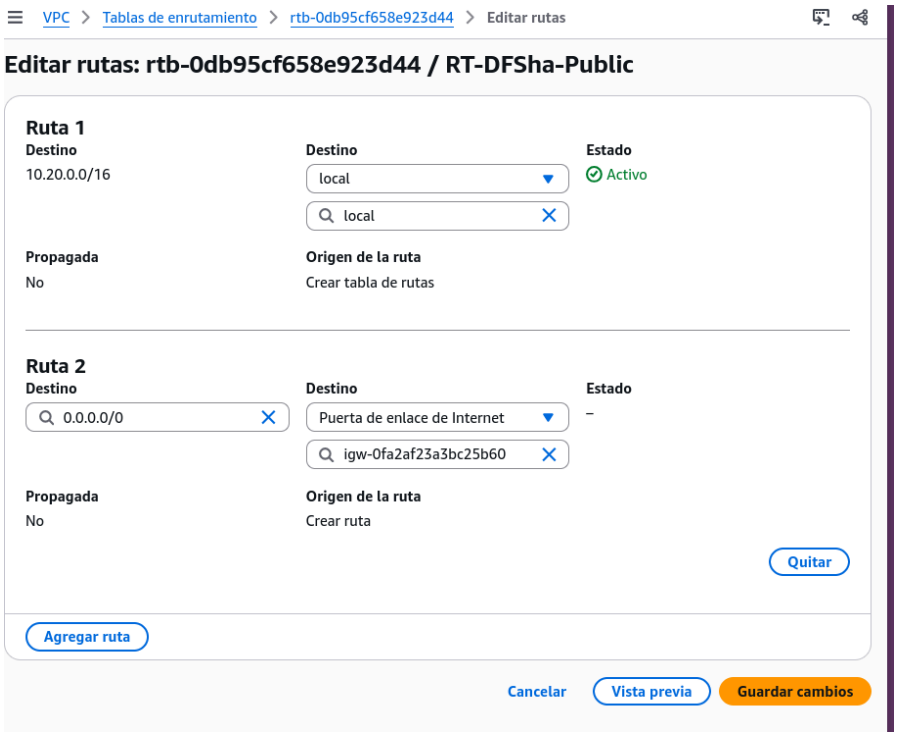


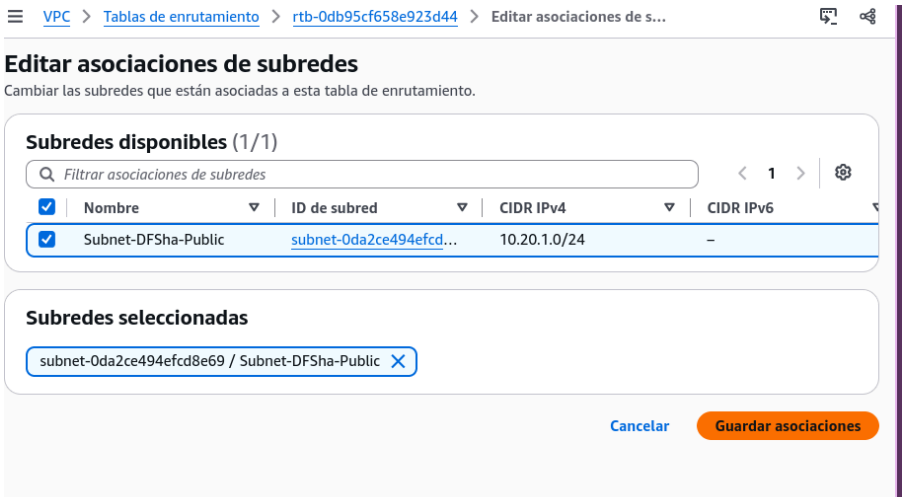
Se crea el gateway de internet y se le asocia a la vpc.



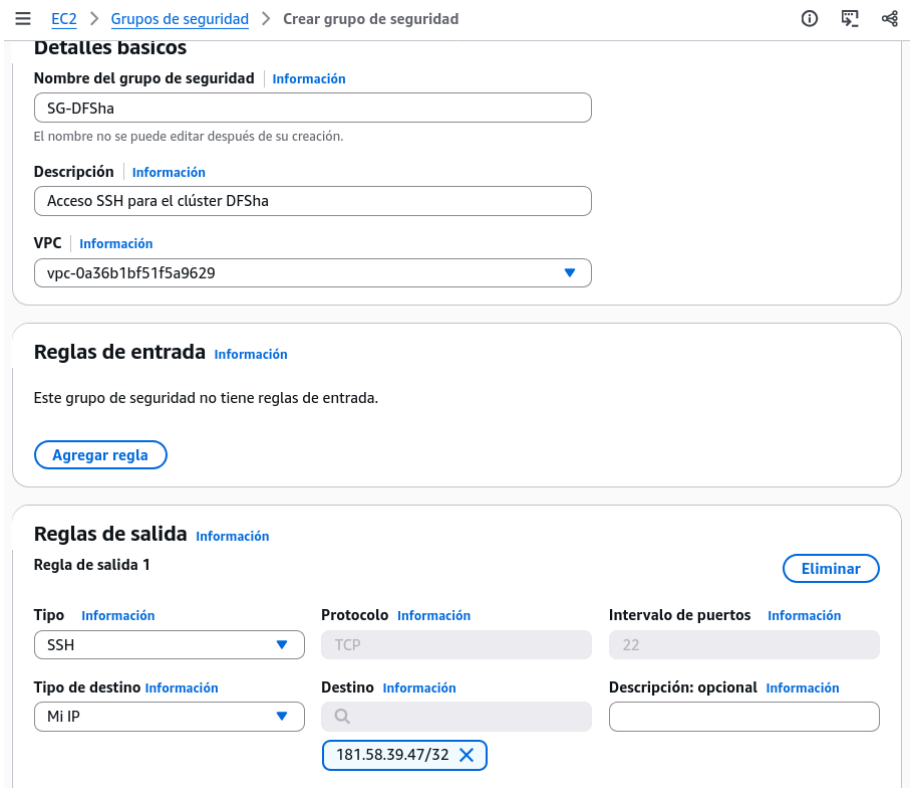


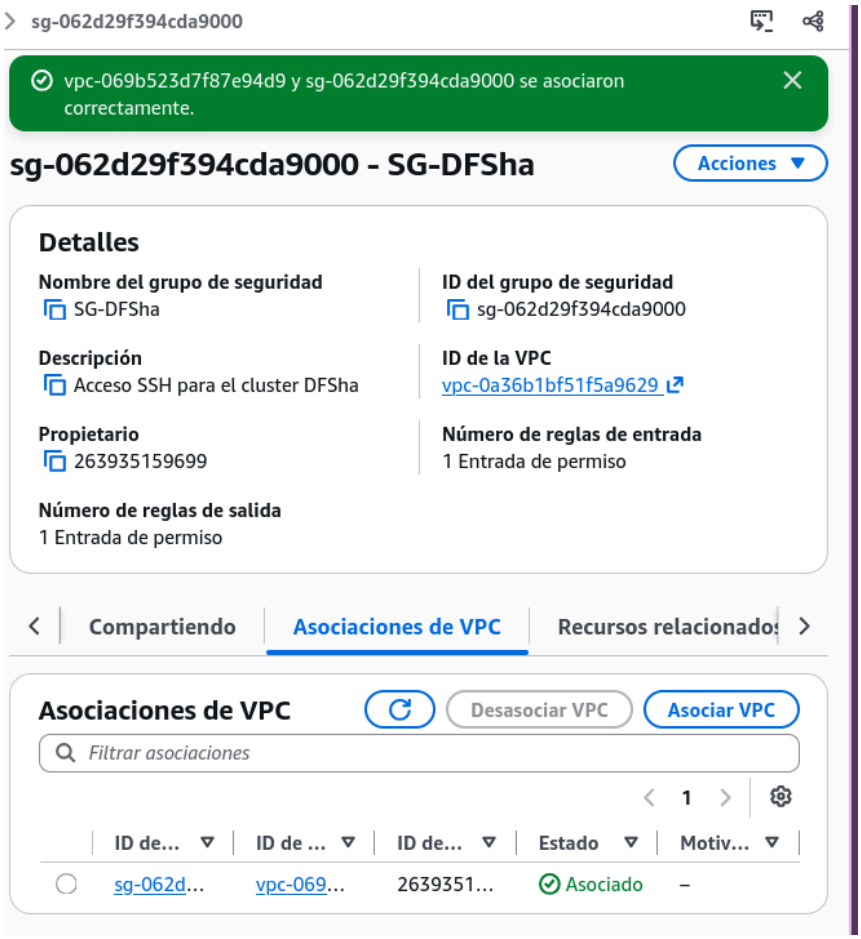
Se crea la tabla de enrutamiento y se le asocia la ruta al gateway y se asociada la subred.



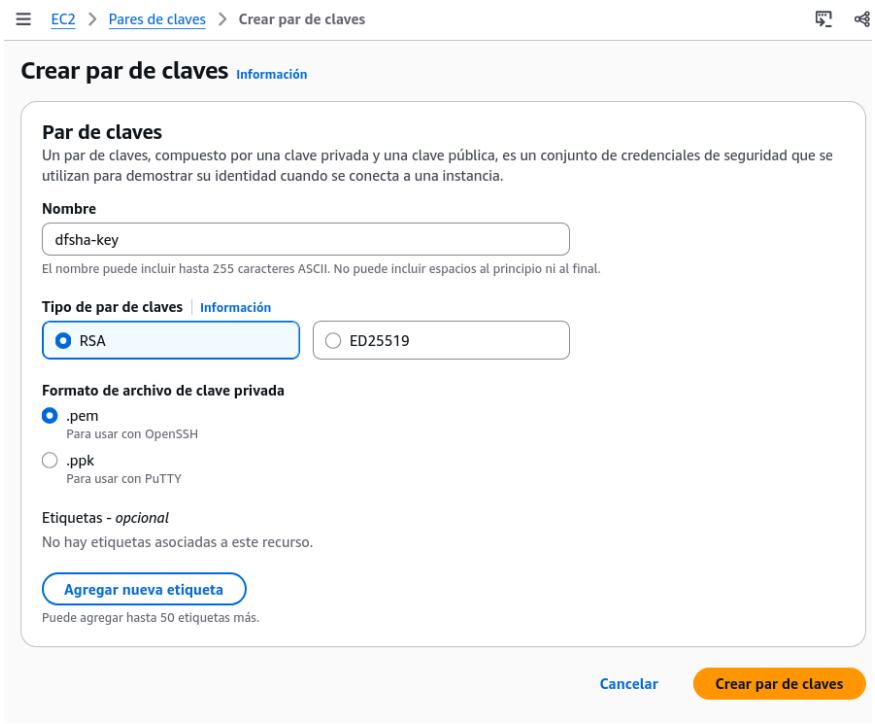


Se crea un grupo de seguridad asociado a la vpc creada, con la regla de entrada ssh.





Se crean los pares de claves.



8.1 Entorno de despliegue en AWS Academy

Se crea una instancia con Ubuntu 24.0, un tipo t2.large y todas las configuraciones anteriores:



Tipo de instancia Información | Obtener asesoramiento

Tipo de instancia

t2.large
Familia: t2 2 vCPU 8 GiB Memoria Generación actual: true
Bajo demanda Windows base precios: 0.1208 USD por hora
Bajo demanda Ubuntu Pro base precios: 0.0963 USD por hora
Bajo demanda RHEL base precios: 0.1216 USD por hora
Bajo demanda SUSE base precios: 0.1928 USD por hora
Bajo demanda Linux base precios: 0.0928 USD por hora

Todas las generaciones

Comparar tipos de instancias

Se aplican costos adicionales a las AMI con software preinstalado

Par de claves (inicio de sesión) Información

Puede utilizar un par de claves para conectarse de forma segura a la instancia. Asegúrese de que tiene acceso al par de claves seleccionado antes de lanzar la instancia.

Nombre del par de claves - *obligatorio*

dfsha-key

Crear un nuevo par de claves

Configuraciones de red Información

Editar

Red | Información
vpc-0a36b1bf51f5a9629

Subred | Información
No hay preferencia (subred predeterminada en cualquier zona de disponibilidad)

Asignación automática de IP pública | Información
Habilitar

Se aplican cargos adicionales cuando no se cumplen los límites del nivel gratuito

Firewall (grupos de seguridad) | Información
Un grupo de seguridad es un conjunto de reglas de firewall que controlan el tráfico de la instancia. Agregue reglas para permitir que un tráfico específico llegue a la instancia.

- ○ Crear un grupo de seguridad
- ◉ Seleccionar un grupo de seguridad existente

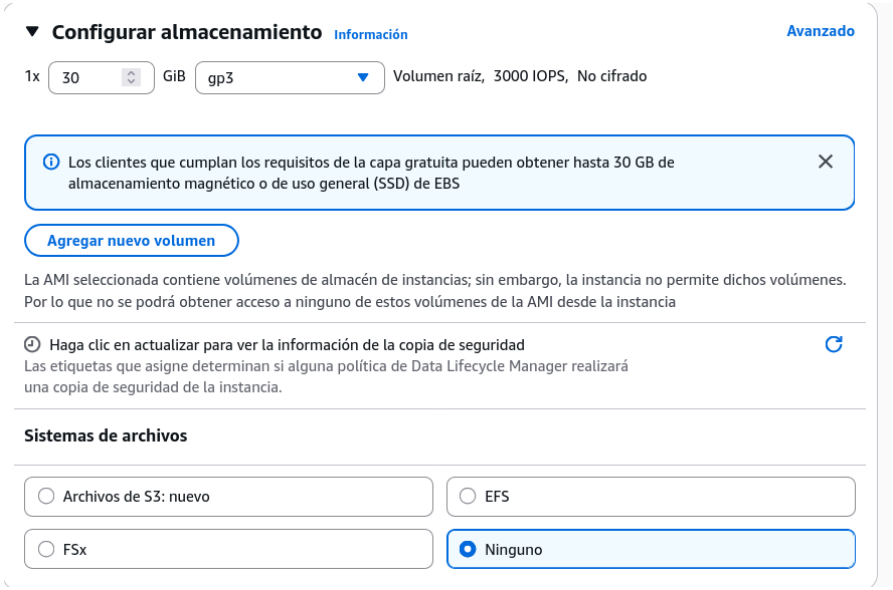
Grupos de seguridad comunes | Información

Seleccionar grupos de seguridad

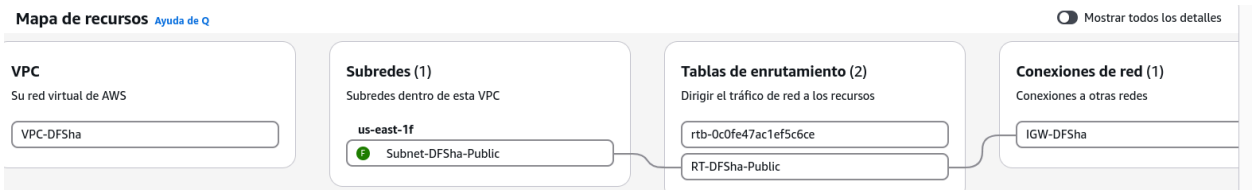
SG-DFSha sg-062d29f394cda9000
VPC: vpc-0a36b1bf51f5a9629

Comparar reglas de grupo de seguridad

Los grupos de seguridad que agrega o elimine aquí se agregarán a todas las interfaces de red o se eliminarán de ellas.



Al final queda:



8.3 Instrucciones de instalación

Dentro de la instancia se actualiza mediante apt update y se instalan las siguientes dependencias:

```
sudo apt install -y \
    git \
    curl \
    wget \
    unzip \
    openssl \
    ca-certificates \
    gnupg \
    lsb-release \
    python3 \
    python3-venv \
    python3-pip
```

```
docker-ce \
docker-ce-cli \
containerd.io \
docker-buildx-plugin \
docker-compose-plugin
```

Luego se clona el repositorio y se comprueba.

```
ubuntu@ip-172-31-27-218:~$ git clone https://github.com/SoftwareArchitecture-DistributedSystems/DFSha.git
Cloning into 'DFSha'...
remote: Enumerating objects: 187, done.
remote: Counting objects: 100% (187/187), done.
remote: Compressing objects: 100% (108/108), done.
remote: Total 187 (delta 71), reused 187 (delta 71), pack-reused 0 (from 0)
Receiving objects: 100% (187/187), 60.28 KiB | 15.07 MiB/s, done.
Resolving deltas: 100% (71/71), done.
ubuntu@ip-172-31-27-218:~$ ls
DFSha
ubuntu@ip-172-31-27-218:~$ cd DFSha/
ubuntu@ip-172-31-27-218:~/DFSha$ ls -la
total 68
drwxrwxr-x 12 ubuntu docker 4096 Nov  8 21:03 .
drwxr-x---  5 ubuntu ubuntu 4096 Nov  8 21:03 ..
-rw-rw-r--  1 ubuntu docker  199 Nov  8 21:03 .env.example
drwxrwxr-x  8 ubuntu docker 4096 Nov  8 21:03 .git
-rw-rw-r--  1 ubuntu docker  512 Nov  8 21:03 .gitignore
drwxrwxr-x  2 ubuntu docker 4096 Nov  8 21:03 client
drwxrwxr-x  3 ubuntu docker 4096 Nov  8 21:03 data-node
-rw-rw-r--  1 ubuntu docker 3881 Nov  8 21:03 docker-compose.yml
drwxrwxr-x  3 ubuntu docker 4096 Nov  8 21:03 docs
drwxrwxr-x  2 ubuntu docker 4096 Nov  8 21:03 logs
drwxrwxr-x  3 ubuntu docker 4096 Nov  8 21:03 metadata-server
drwxrwxr-x  3 ubuntu docker 4096 Nov  8 21:03 proto
-rw-rw-r--  1 ubuntu docker   27 Nov  8 21:03 requirements-dev.txt
-rw-rw-r--  1 ubuntu docker   70 Nov  8 21:03 requirements.txt
drwxrwxr-x  2 ubuntu docker 4096 Nov  8 21:03 scripts
drwxrwxr-x  2 ubuntu docker 4096 Nov  8 21:03 shared
drwxrwxr-x  7 ubuntu docker 4096 Nov  8 21:03 tests
```

Se ejecuta el script para inciar el cluster y descargar las imagenes, crear la red, los volumenes y los contenedores de cada nodo.

```
ubuntu@ip-172-31-27-218:~/DFSha$ scripts/start_cluster.sh
certificados en certs/<nombre>/{node.crt,node.key,ca.crt}
[+] Building 34.6s (47/47) FINISHED
 => [internal] load local bake definitions
 => => reading from stdin 3.67kB
 => [ms1 internal] load build definition from Dockerfile
 => => transferring dockerfile: 594B
 => [dn5 internal] load build definition from Dockerfile
 => => transferring dockerfile: 530B
 => [client internal] load build definition from Dockerfile
 => => transferring dockerfile: 710B
 => [dn5 internal] load metadata for docker.io/library/python:3.11-slim
 => [ms3 internal] load .dockerignore
 => => transferring context: 2B
 => [dn5 1/8] FROM docker.io/library/python:3.11-slim@sha256:07d4824036af339fa1b6ef769cc99d9c10a
 => => resolve docker.io/library/python:3.11-slim@sha256:07d4824036af339fa1b6ef769cc99d9c10a1417
 => => sha256:bbe7053e6def9842d628414346e26f25e499463adb6ca48c0c8932e036e44000 249B / 249B
 => => sha256:53daecf7856e9b7235583f1ec326387aae7d29af3d5cd2efc20246259ab817cc 1.29MB / 1.29MB
 => => sha256:7efbce397249e9202b070b8e638baff71160948c640b9a44f2226419a71f410d 14.46MB / 14.46MB
 => => sha256:37fb86c2a767b88d500c6872195f88078f64765fe0d2f9cc375cc2efbd4c0a41 29.86MB / 29.86MB
```

```
[+] up 28/28
 ✓ Image dfsha-dn3          Built                                                                                    35.6
 ✓ Image dfsha-ms3          Built                                                                                    34.6
 ✓ Image dfsha-dn5          Built                                                                                    34.8
 ✓ Image dfsha-dn4          Built                                                                                    35.6
 ✓ Image dfsha-dn2          Built                                                                                    35.6
 ✓ Image dfsha-ms2          Built                                                                                    35.1
 ✓ Image dfsha-ms1          Built                                                                                    34.9
 ✓ Image dfsha-client       Built                                                                                    34.9
 ✓ Image dfsha-dn1          Built                                                                                    34.7
 ✓ Network dfsha_dfsha      Created                                                                                   0.1
 ✓ Volume dfsha_ms3-data    Created                                                                                   0.0
 ✓ Volume dfsha_client-home Created                                                                                   0.0
 ✓ Volume dfsha_dn4-data    Created                                                                                   0.1
 ✓ Volume dfsha_dn1-data    Created                                                                                   0.1
 ✓ Volume dfsha_dn2-data    Created                                                                                   0.0
 ✓ Volume dfsha_dn5-data    Created                                                                                   0.1
 ✓ Volume dfsha_dn3-data    Created                                                                                   0.1
 ✓ Volume dfsha_ms1-data    Created                                                                                   0.1
 ✓ Volume dfsha_ms2-data    Created                                                                                   0.0
 ✓ Container dfsha-ms3-1    Started                                                                                   4.9
 ✓ Container dfsha-ms2-1    Started                                                                                   3.8
 ✓ Container dfsha-ms1-1    Started                                                                                   3.9
 ✓ Container dfsha-dn5-1    Started                                                                                   1.4
 ✓ Container dfsha-dn1-1    Started                                                                                   2.1
 ✓ Container dfsha-dn2-1    Started                                                                                   2.1
 ✓ Container dfsha-client-1 Started                                                                                   2.0
 ✓ Container dfsha-dn3-1    Started                                                                                   1.9
 ✓ Container dfsha-dn4-1    Started                                                                                   1.1
clúster en marcha. Cliente: docker compose exec client python client/main.py login <usuario>
```

```
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose ps
NAME            IMAGE                                                                                  COMMAND                  SERVICE   CREATED         STATUS
 PORTS
dfsha-client-1  sha256:0d6fb8321fe0f28c0c3f7332fbedc61ab4dd013a6fea3ca3ea19b5622f4548cc   "sleep infinity"         client    3 minutes ago   Up 3 minutes

dfsha-dn1-1     sha256:9d6b61d1d4428ee028b2220c44ad763b9ff2abb4ba44c752e6804a2da8705932   "python data-node/ma…"   dn1       3 minutes ago   Up 3 minutes

dfsha-dn2-1     sha256:e33b8c972ba21026ba91b20e02fafbfc93089d4ef98d2ae2ff5d1a4c58701cca   "python data-node/ma…"   dn2       3 minutes ago   Up 3 minutes

dfsha-dn3-1     sha256:7c5520d431e521f506504988bdf04505fbb4a589b85d4f21d0ae0417a473b992   "python data-node/ma…"   dn3       3 minutes ago   Up 3 minutes

dfsha-dn4-1     sha256:565749dd10dceb7aec45ef2e5bc1afbcfd08f60778c38f5637ea375169a98540   "python data-node/ma…"   dn4       3 minutes ago   Up 3 minutes

dfsha-dn5-1     sha256:b26e800a8a40f29e8506be3131270c0bd19c853f3d86dc74449ba2992778c2a4   "python data-node/ma…"   dn5       3 minutes ago   Up 3 minutes

dfsha-ms1-1     sha256:ba26c47806d54f4af543ca2be1c7b6decb28fe477d32192879d0945d52e235a4   "python metadata-ser…"   ms1       3 minutes ago   Up 3 minutes

dfsha-ms2-1     sha256:891933b183d144ca63985e62127f2d65631ae1745d476d7ddf356f4ed05eec0f   "python metadata-ser…"   ms2       3 minutes ago   Up 3 minutes

dfsha-ms3-1     sha256:1b2fd8c5b0d2661533ef7723150567eaafbe8b4a92fdcd8781bf0c2851b7f577   "python metadata-ser…"   ms3       3 minutes ago   Up 3 minutes
```

8.4 Máquinas virtuales, nodos o contenedores

Se inician los servicios con el compose up.

```
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose up -d --build
[+] Building 1.8s (47/47) FINISHED
 => [internal] load local bake definitions                                                                                0.0s
 => => reading from stdin 3.67kB                                                                                          0.0s
 => [dn3 internal] load build definition from Dockerfile                                                                  0.0s
 => => transferring dockerfile: 530B                                                                                      0.0s
 => [ms1 internal] load build definition from Dockerfile                                                                  0.0s
 => => transferring dockerfile: 594B                                                                                      0.0s
 => [client internal] load build definition from Dockerfile                                                               0.0s
 => => transferring dockerfile: 710B                                                                                      0.0s
 => [dn2 internal] load metadata for docker.io/library/python:3.11-slim                                                   0.1s
 => [dn5 internal] load .dockerignore                                                                                     0.0s
 => => transferring context: 2B                                                                                           0.0s
 => [ms2 1/8] FROM docker.io/library/python:3.11-slim@sha256:07d4824036af339fa1b6ef769cc99d9c10a141769538b52b3b597c84f7a7b535  0.0s
 => => resolve docker.io/library/python:3.11-slim@sha256:07d4824036af339fa1b6ef769cc99d9c10a141769538b52b3b597c84f7a7b535  0.0s
 => [ms3 internal] load build context                                                                                     0.0s
```

```
 => [dn1] resolving provenance for metadata file                                                                          0.0s
[+] up 18/18
 ✓ Image dfsha-dn1          Built                                                                                         2.
 ✓ Image dfsha-dn2          Built                                                                                         2.
 ✓ Image dfsha-dn3          Built                                                                                         2.
 ✓ Image dfsha-ms1          Built                                                                                         2.
 ✓ Image dfsha-dn5          Built                                                                                         2.
 ✓ Image dfsha-ms2          Built                                                                                         1.
 ✓ Image dfsha-client       Built                                                                                         2.
 ✓ Image dfsha-dn4          Built                                                                                         1.
 ✓ Image dfsha-ms3          Built                                                                                         2.
 ✓ Container dfsha-dn3-1    Running                                                                                       0.
 ✓ Container dfsha-ms2-1    Running                                                                                       0.
 ✓ Container dfsha-dn2-1    Running                                                                                       0.
 ✓ Container dfsha-ms3-1    Running                                                                                       0.
 ✓ Container dfsha-ms1-1    Running                                                                                       0.
 ✓ Container dfsha-dn1-1    Running                                                                                       0.
 ✓ Container dfsha-dn5-1    Running                                                                                       0.
 ✓ Container dfsha-client-1 Running                                                                                       0.
 ✓ Container dfsha-dn4-1    Running                                                                                       0.
```

8.4 Configuración de servicios

Se revisan los logs para comprobar que todos los contenedores estén bien

```
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose logs --tail=50 ms1 ms2 ms3
ms1-1  | 2026-11-08 21:09:35,218 ms MS ms1:7000 listo (gRPC 7100)
ms1-1  | I1108 21:14:35.240320       7 fork_posix.cc:71] Other threads are currently calling into gRPC, skipping fork() handlers
ms2-1  | 2026-11-08 21:09:35,673 ms MS ms2:7000 listo (gRPC 7100)
ms2-1  | I1108 21:14:35.698844       7 fork_posix.cc:71] Other threads are currently calling into gRPC, skipping fork() handlers
ms3-1  | 2026-11-08 21:09:35,870 ms MS ms3:7000 listo (gRPC 7100)
ms3-1  | I1108 21:14:35.855500       7 fork_posix.cc:71] Other threads are currently calling into gRPC, skipping fork() handlers
```

```
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose logs --tail=50 ms1 ms2 ms3
ms1-1  | 2026-11-08 21:09:35,218 ms MS ms1:7000 listo (gRPC 7100)
ms1-1  | I1108 21:14:35.240320       7 fork_posix.cc:71] Other threads are currently calling into gRPC, skipping fork() handlers
ms2-1  | 2026-11-08 21:09:35,673 ms MS ms2:7000 listo (gRPC 7100)
ms2-1  | I1108 21:14:35.698844       7 fork_posix.cc:71] Other threads are currently calling into gRPC, skipping fork() handlers
ms3-1  | 2026-11-08 21:09:35,870 ms MS ms3:7000 listo (gRPC 7100)
ms3-1  | I1108 21:14:35.855500       7 fork_posix.cc:71] Other threads are currently calling into gRPC, skipping fork() handlers
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose logs --tail=50 dn1 dn2 dn3 dn4 dn5
dn2-1  | 2026-11-08 21:09:36,752 dn DN dn2:7001 listo
dn4-1  | 2026-11-08 21:09:36,245 dn DN dn4:7001 listo
dn3-1  | 2026-11-08 21:09:36,491 dn DN dn3:7001 listo
dn1-1  | 2026-11-08 21:09:36,913 dn DN dn1:7001 listo
dn5-1  | 2026-11-08 21:09:35,991 dn DN dn5:7001 listo
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose logs -f
dn5-1  | 2026-11-08 21:09:35,991 dn DN dn5:7001 listo
dn2-1  | 2026-11-08 21:09:36,752 dn DN dn2:7001 listo
dn1-1  | 2026-11-08 21:09:36,913 dn DN dn1:7001 listo
dn4-1  | 2026-11-08 21:09:36,245 dn DN dn4:7001 listo
ms1-1  | 2026-11-08 21:09:35,218 ms MS ms1:7000 listo (gRPC 7100)
ms1-1  | I1108 21:14:35.240320       7 fork_posix.cc:71] Other threads are currently calling into gRPC, skipping fork() handlers
dn3-1  | 2026-11-08 21:09:36,491 dn DN dn3:7001 listo
ms3-1  | 2026-11-08 21:09:35,870 ms MS ms3:7000 listo (gRPC 7100)
ms3-1  | I1108 21:14:35.855500       7 fork_posix.cc:71] Other threads are currently calling into gRPC, skipping fork() handlers
ms2-1  | 2026-11-08 21:09:35,673 ms MS ms2:7000 listo (gRPC 7100)
ms2-1  | I1108 21:14:35.698844       7 fork_posix.cc:71] Other threads are currently calling into gRPC, skipping fork() handlers
```

Se ejecuta el python del cliente para registrar un usuario e iniciar sesión, luego se crea un directorio y se comprueba su contenido.

```
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client python client/main.py register demo
contraseña:
ok
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client python client/main.py login demo
contraseña:
ok
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client python client/main.py mkdir documentos
ok
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client python client/main.py mkdir documentos/pruebas
ok
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client python client/main.py ls
d            0 v0 documentos
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client python client/main.py cd documentos
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client python client/main.py pwd
/users/demo/documentos
ubuntu@ip-172-31-27-218:~/DFSha$
```

8.6 Instrucciones de ejecución

Se genera un archivo de 20 MB dentro del contenedor cliente y se sube a DFSha.

```
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client python client/main.py mkdir documentos
ok
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client python client/main.py mkdir documentos/pruebas
ok
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client python client/main.py ls
d            0 v0 documentos
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client python client/main.py cd documentos
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client python client/main.py pwd
/users/demo/documentos
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client sh -c \
'head -c 20971520 /dev/urandom > /tmp/prueba.bin'
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client \
python client/main.py put /tmp/prueba.bin pruebas.bin
21.0 MB en 1.44s (14.5 MB/s)
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client python client/main.py ls
d            0 v0 pruebas
-     20971520 v1 pruebas.bin
```

8.7 Verificación de la comunicación entre nodos

Descargar el archivo y se Compara los hashes para saber si el archivo fue transferido al sistema distribuido y posteriormente recuperado sin alterar su contenido.

```
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client sh -c \
'sha256sum /tmp/prueba.bin /tmp/recuperado.bin'
215101414c7f7b1fe4093c46470329f1362946e8ddc6e97a4e673ba793eaafb6  /tmp/prueba.bin
215101414c7f7b1fe4093c46470329f1362946e8ddc6e97a4e673ba793eaafb6  /tmp/recuperado.bin
```

9. Pruebas y validación

El archivo de 20 MB debe dividirse en tres bloques:

- Bloque 1: 8 MiB.
- Bloque 2: 8 MiB.
- Bloque 3: el contenido restante.

Cada bloque debe contar con tres réplicas en nodos diferentes.

```
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec -w /app/client client python -c '
from dfs_client import DFSClient

c = DFSClient()
path = c.abspath("pruebas.bin")
info = c.meta.get_block_locations(path)

print("Archivo:", path)
print("Versión:", info.file_version)
print("Tamaño:", info.total_size)
print("Número de bloques:", len(info.blocks))

for b in info.blocks:
    print("Bloque:", b.block_id)
    print("Réplicas:", list(b.replicas))
    print("Cantidad:", len(b.replicas))
    print("---")
'
Archivo: /users/demo/documentos/pruebas.bin
Versión: 1
Tamaño: 20971520
Número de bloques: 3
Bloque: c49c1965-ef68-4353-9ed8-499fed45cdf6
Réplicas: ['dn5:7001', 'dn1:7001', 'dn4:7001']
Cantidad: 3
---
Bloque: ac12603b-f932-4089-b542-cf3c36dd10be
Réplicas: ['dn1:7001', 'dn2:7001', 'dn3:7001']
Cantidad: 3
---
Bloque: 6ac03aa9-a081-4686-bc18-4df5a5a1e8b1
Réplicas: ['dn2:7001', 'dn4:7001', 'dn3:7001']
Cantidad: 3
---
```

Se prueba parando un nodo y se verifica si el sistema es capaz de continuar su distribución sin él.

```
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose stop dn1
[+] stop 1/1
 ✓ Container dfsha-dn1-1 Stopped                                                                                    10.1s
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose ps
NAME            IMAGE                                                                                  COMMAND                  SERVICE   CREATED         STATUS
 PORTS
dfsha-client-1   sha256:d177b577c452b483cc89fe475a6c0a54e0cdb5acf519f5003d1950bcc542ace2   "sleep infinity"         client    9 minutes ago   Up 9 minutes

dfsha-dn2-1      sha256:7f95f54cacb67b5d46a8ce34b131cf1f4de2e689a905d968617d3bad9c593e84   "python data-node/ma…"   dn2       9 minutes ago   Up 9 minutes

dfsha-dn3-1      sha256:8ba6e4360fc717c805a62ee0cfd6fd2e3af9ec3453fec9cb661506dae83d8329   "python data-node/ma…"   dn3       9 minutes ago   Up 9 minutes

dfsha-dn4-1      sha256:c330986433df63b67feacc3cd52258c45b7fe75ef9f943a81d9ea7867c691283   "python data-node/ma…"   dn4       9 minutes ago   Up 9 minutes

dfsha-dn5-1      sha256:f7522cf04c61a4ea828f75dd1f509931590f5c8b36c06f4c15a2a41fe7940c24   "python data-node/ma…"   dn5       9 minutes ago   Up 9 minutes

dfsha-ms1-1      sha256:9daa27008583cd0ed359f18ba4fd930dd6cce455ab84a5a0d914cc3a93c1bff3   "python metadata-ser…"   ms1       9 minutes ago   Up 9 minutes

dfsha-ms2-1      sha256:3696ddd9bbf3e42c5cfef7491fc17076a0269a6508c881aaf9fa00d9e210447b   "python metadata-ser…"   ms2       9 minutes ago   Up 9 minutes

dfsha-ms3-1      sha256:423cd807e7814f0eb7586e93916a61387fe0fd9c025d47670425c5d736941d1b   "python metadata-ser…"   ms3       9 minutes ago   Up 9 minutes
```

```
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client \
python client/main.py get pruebas.bin /tmp/recuperado_fallo.bin
21.0 MB en 0.20s (105.3 MB/s)
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client sh -c \
'sha256sum /tmp/prueba.bin /tmp/recuperado_fallo.bin'
215101414c7f7b1fe4093c46470329f1362946e8ddc6e97a4e673ba793eaafb6  /tmp/prueba.bin
215101414c7f7b1fe4093c46470329f1362946e8ddc6e97a4e673ba793eaafb6  /tmp/recuperado_fallo.bin
ubuntu@ip-172-31-27-218:~/DFSha$
```

```
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec -w /app/client client python -c '
from dfs_client import DFSClient

c = DFSClient()
path = c.abspath("pruebas.bin")
info = c.meta.get_block_locations(path)

print("Archivo:", path)
print("Versión:", info.file_version)
print("Tamaño:", info.total_size)
print("Número de bloques:", len(info.blocks))

for b in info.blocks:
    print("Bloque:", b.block_id)
    print("Réplicas:", list(b.replicas))
    print("Cantidad:", len(b.replicas))
    print("---")
'
Archivo: /users/demo/documentos/pruebas.bin
Versión: 1
Tamaño: 20971520
Número de bloques: 3
Bloque: c49c1965-ef68-4353-9ed8-499fed45cdf6
Réplicas: ['dn2:7001', 'dn5:7001', 'dn4:7001']
Cantidad: 3
---
Bloque: ac12603b-f932-4089-b542-cf3c36dd10be
Réplicas: ['dn3:7001', 'dn4:7001', 'dn2:7001']
Cantidad: 3
---
Bloque: 6ac03aa9-a081-4686-bc18-4df5a5a1e8b1
Réplicas: ['dn2:7001', 'dn3:7001', 'dn4:7001']
Cantidad: 3
---
```

```
 ✓ Container dfsha-dn1-1 Started                                                                                                    0.2s
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose ps
NAME            IMAGE                                                                  COMMAND                  SERVICE   CREATED          STATUS
   PORTS
dfsha-client-1  sha256:d177b577c452b483cc89fe475a6c0a54e0cdb5acf519f5003d1950bcc542ace2  "sleep infinity"         client    12 minutes ago   Up 12 minutes

dfsha-dn1-1     sha256:26457cacc25b9bff9deed6a8b67c0a22099df3cf3d01e918b4e8a45b665b43f8  "python data-node/ma…"   dn1       12 minutes ago   Up 2 seconds

dfsha-dn2-1     sha256:7f95f54cacb67b5d46a8ce34b131cf1f4de2e689a905d968617d3bad9c593e84  "python data-node/ma…"   dn2       12 minutes ago   Up 12 minutes

dfsha-dn3-1     sha256:8ba6e4360fc717c805a62ee0cfd6fd2e3af9ec3453fec9cb661506dae83d8329  "python data-node/ma…"   dn3       12 minutes ago   Up 12 minutes

dfsha-dn4-1     sha256:c330986433df63b67feacc3cd52258c45b7fe75ef9f943a81d9ea7867c691283  "python data-node/ma…"   dn4       12 minutes ago   Up 12 minutes

dfsha-dn5-1     sha256:f7522cf04c61a4ea828f75dd1f509931590f5c8b36c06f4c15a2a41fe7940c24  "python data-node/ma…"   dn5       12 minutes ago   Up 12 minutes

dfsha-ms1-1     sha256:9daa27008583cd0ed359f18ba4fd930dd6cce455ab84a5a0d914cc3a93c1bff3  "python metadata-ser…"   ms1       12 minutes ago   Up 12 minutes

dfsha-ms2-1     sha256:3696ddd9bbf3e42c5cfef7491fc17076a0269a6508c881aaf9fa00d9e210447b  "python metadata-ser…"   ms2       12 minutes ago   Up 12 minutes

dfsha-ms3-1     sha256:423cd807e7814f0eb7586e93916a61387fe0fd9c025d47670425c5d736941d1b  "python metadata-ser…"   ms3       12 minutes ago   Up 12 minutes
```

Se verifica el líder y se para para saber si el sistema puede continuar sin él.

```
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client python -c '
import sys
sys.path.insert(0, "/app/client")

from dfs_client import DFSClient

c = DFSClient()
c.meta.list_dir(c.abspath("."))
print("Lider detectado:", c.meta.router.leader)
'
Lider detectado: ms2:7100
```

```
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose stop ms2
[+] stop 1/1
 ✓ Container dfsha-ms2-1 Stopped                                                                                    10.1s
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client \
python client/main.py mkdir recuperacion_raft
ok
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client \
python client/main.py ls
d            0 v0 documentos
d            0 v0 recuperacion_raft
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose start ms2
[+] start 1/1
 ✓ Container dfsha-ms2-1 Started                                                                                     0.2s
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose ps
NAME             IMAGE                                                                COMMAND                  SERVICE   CREATED          STATUS
   PORTS
dfsha-client-1   sha256:d177b577c452b483cc89fe475a6c0a54e0cdb5acf519f5003d1950bcc542ace2   "sleep infinity"         client    21 minutes ago   Up 21 minutes

dfsha-dn1-1      sha256:26457cacc25b9bff9deed6a8b67c0a22099df3cf3d01e918b4e8a45b665b43f8   "python data-node/ma…"   dn1       21 minutes ago   Up 8 minutes

dfsha-dn2-1      sha256:7f95f54cacb67b5d46a8ce34b131cf1f4de2e689a905d968617d3bad9c593e84   "python data-node/ma…"   dn2       21 minutes ago   Up 21 minutes

dfsha-dn3-1      sha256:8ba6e4360fc717c805a62ee0cfd6fd2e3af9ec3453fec9cb661506dae83d8329   "python data-node/ma…"   dn3       21 minutes ago   Up 21 minutes

dfsha-dn4-1      sha256:c330986433df63b67feacc3cd52258c45b7fe75ef9f943a81d9ea7867c691283   "python data-node/ma…"   dn4       21 minutes ago   Up 21 minutes

dfsha-dn5-1      sha256:f7522cf04c61a4ea828f75dd1f509931590f5c8b36c06f4c15a2a41fe7940c24   "python data-node/ma…"   dn5       21 minutes ago   Up 21 minutes

dfsha-ms1-1      sha256:9daa27008583cd0ed359f18ba4fd930dd6cce455ab84a5a0d914cc3a93c1bff3   "python metadata-ser…"   ms1       21 minutes ago   Up 21 minutes

dfsha-ms2-1      sha256:3696ddd9bbf3e42c5cfef7491fc17076a0269a6508c881aaf9fa00d9e210447b   "python metadata-ser…"   ms2       21 minutes ago   Up 2 seconds

dfsha-ms3-1      sha256:423cd807e7814f0eb7586e93916a61387fe0fd9c025d47670425c5d736941d1b   "python metadata-ser…"   ms3       21 minutes ago   Up 21 minutes
```

```
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec -T client sh -c \
'printf "Contenido de prueba RF3" | python client/main.py write documentos/rf3.txt'
ok
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose exec client \
python client/main.py read documentos/rf3.txt 0 23
Contenido de prueba RF3ubuntu@ip-172-31-27-218:~/DFSha$
```

9.1 Entorno y metodología de pruebas

Las pruebas se ejecutan con pytest desde la propia VM de AWS contra el clúster desplegado con Docker Compose (3 servidores de metadatos, 5 nodos de datos y cliente, con mTLS). La VM es una instancia EC2 t2.large con Ubuntu 24.04 (2 vCPU, 8 GiB de RAM y disco gp3 de 30 GiB) en us-east-1. Las fixtures están en tests/conftest.py y las utilidades en tests/helpers.py; las pruebas de fallos detienen y arrancan contenedores con docker compose. La corrida local (sección 7.10) se conserva como referencia..

Se bajan los contenedores y se prepara el entorno de Python configurando las variables locales e instalando las dependencias necesarias.

```
ubuntu@ip-172-31-27-218:~/DFSha$ docker compose down
[+] down 10/10
 ✓ Container dfsha-dn2-1    Removed
 ✓ Container dfsha-dn3-1    Removed
 ✓ Container dfsha-dn1-1    Removed
 ✓ Container dfsha-dn4-1    Removed
 ✓ Container dfsha-client-1 Removed
 ✓ Container dfsha-dn5-1    Removed
 ✓ Container dfsha-ms3-1    Removed
 ✓ Container dfsha-ms1-1    Removed
 ✓ Container dfsha-ms2-1    Removed
 ✓ Network dfsha_dfsha      Removed
ubuntu@ip-172-31-27-218:~/DFSha$ python3 -m venv .venv
ubuntu@ip-172-31-27-218:~/DFSha$ source .venv/bin/activate
(.venv) ubuntu@ip-172-31-27-218:~/DFSha$ pip install --upgrade pip
pip install -r requirements-dev.txt
Requirement already satisfied: pip in ./.venv/lib/python3.12/site-packages (24.0)
Collecting pip
  Downloading pip-26.3-py3-none-any.whl.metadata (4.6 kB)
Downloading pip-26.3-py3-none-any.whl (2.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.0/2.0 MB 11.4 MB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 24.0
```

```
.venv) ubuntu@ip-172-31-27-218:~/DFSha$ export DFSHA_LOCAL=1
.venv) ubuntu@ip-172-31-27-218:~/DFSha$ export DFSHA_RUN=/tmp/dfsha_run
.venv) ubuntu@ip-172-31-27-218:~/DFSha$ export PYTHONPATH=$PWD:$PWD/proto/generated:$PWD/client
.venv) ubuntu@ip-172-31-27-218:~/DFSha$ export DFSHA_MS=127.0.0.1:7100,127.0.0.1:7101,127.0.0.1:7102
.venv) ubuntu@ip-172-31-27-218:~/DFSha$ ls -l /tmp/dfsha_run
otal 96
rwxrwxr-x 2 ubuntu docker 4096 Nov  8 22:03 dn1
rw-rw-r-- 1 ubuntu docker   51 Nov  8 22:03 dn1.log
rw-rw-r-- 1 ubuntu docker    6 Nov  8 22:03 dn1.pid
rwxrwxr-x 2 ubuntu docker 4096 Nov  8 22:03 dn2
rw-rw-r-- 1 ubuntu docker   51 Nov  8 22:03 dn2.log
rw-rw-r-- 1 ubuntu docker    6 Nov  8 22:03 dn2.pid
rwxrwxr-x 2 ubuntu docker 4096 Nov  8 22:03 dn3
rw-rw-r-- 1 ubuntu docker   51 Nov  8 22:03 dn3.log
rw-rw-r-- 1 ubuntu docker    6 Nov  8 22:03 dn3.pid
rwxrwxr-x 2 ubuntu docker 4096 Nov  8 22:03 dn4
rw-rw-r-- 1 ubuntu docker   51 Nov  8 22:03 dn4.log
rw-rw-r-- 1 ubuntu docker    6 Nov  8 22:03 dn4.pid
rwxrwxr-x 2 ubuntu docker 4096 Nov  8 22:03 dn5
rw-rw-r-- 1 ubuntu docker   51 Nov  8 22:03 dn5.log
rw-rw-r-- 1 ubuntu docker    6 Nov  8 22:03 dn5.pid
rwxrwxr-x 2 ubuntu docker 4096 Nov  8 22:04 ms0
rw-rw-r-- 1 ubuntu docker   63 Nov  8 22:03 ms0.log
rw-rw-r-- 1 ubuntu docker    6 Nov  8 22:03 ms0.pid
rwxrwxr-x 2 ubuntu docker 4096 Nov  8 22:03 ms1
rw-rw-r-- 1 ubuntu docker   63 Nov  8 22:03 ms1.log
rw-rw-r-- 1 ubuntu docker    6 Nov  8 22:03 ms1.pid
rwxrwxr-x 2 ubuntu docker 4096 Nov  8 22:04 ms2
rw-rw-r-- 1 ubuntu docker   63 Nov  8 22:03 ms2.log
rw-rw-r-- 1 ubuntu docker    6 Nov  8 22:03 ms2.pid
```

Luego se ejecuta el script de las pruebas automatizadas para comprobar cada caso:

Componente Dirección

MS1 127.0.0.1:7100

MS2 127.0.0.1:7101

MS3 127.0.0.1:7102

DN1 127.0.0.1:7201

DN2 127.0.0.1:7202

DN3 127.0.0.1:7203

DN4 127.0.0.1:7204

DN5 127.0.0.1:7205

```
platform linux -- Python 3.12.3, pytest-9.2.0, pluggy-1.7.0 -- /home/ubuntu/DFSha/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/ubuntu/DFSha
collected 25 items

tests/concurrency/test_concurrent_access.py::test_lock_exclusivo_entre_clientes PASSED                                  [  4%]
tests/concurrency/test_concurrent_access.py::test_locks_compartidos_conviven PASSED                                     [  8%]
tests/concurrency/test_concurrent_access.py::test_escritura_de_otro_cliente_sobre_bloque_bloqueado PASSED               [ 12%]
tests/concurrency/test_concurrent_access.py::test_version_conflict_gana_un_solo_escritor PASSED                         [ 16%]
tests/concurrency/test_concurrent_access.py::test_subidas_paralelas PASSED                                              [ 20%]
tests/failures/test_metadata_failure.py::test_caida_del_lider_de_metadatos PASSED                                       [ 24%]
tests/failures/test_node_failure.py::test_lectura_y_escritura_con_un_dn_caido PASSED                                    [ 28%]
tests/failures/test_node_failure.py::test_re_replicacion_tras_caida PASSED                                              [ 32%]
tests/functional/test_directory_operations.py::test_mkdir_ls_rmdir PASSED                                               [ 36%]
tests/functional/test_directory_operations.py::test_duplicados_y_padre_inexistente PASSED                               [ 40%]
tests/functional/test_directory_operations.py::test_cd_es_estado_local PASSED                                           [ 44%]
tests/functional/test_file_operations.py::test_open_write_close_read PASSED                                             [ 48%]
tests/functional/test_file_operations.py::test_reemplazar_crea_nueva_version_y_anexar PASSED                            [ 52%]
tests/functional/test_file_operations.py::test_escritura_no_visible_hasta_close PASSED                                  [ 56%]
tests/functional/test_file_operations.py::test_lector_termina_sobre_la_version_que_abrio PASSED                         [ 60%]
tests/functional/test_file_operations.py::test_rm_y_archivo_inexistente PASSED                                          [ 64%]
tests/functional/test_file_operations.py::test_lectura_por_rango_entre_bloques PASSED                                   [ 68%]
tests/functional/test_file_operations.py::test_aislamiento_entre_usuarios PASSED                                        [ 72%]
tests/functional/test_transfer.py::test_put_get_20mb PASSED                                                             [ 76%]
tests/functional/test_transfer.py::test_archivo_vacio PASSED                                                            [ 80%]
tests/functional/test_transfer.py::test_put_sobre_existente_reemplaza PASSED                                            [ 84%]
tests/performance/test_performance.py::test_throughput
RESULTADO {"MB": 50, "put_MBps": 20.7, "get_MBps": 204.6, "metadatos_ms": 3.2}
PASSED                                                                                                                  [ 88%]
tests/replication/test_integrity.py::test_replica_corrupta_se_descarta_y_se_lee_otra PASSED                             [ 92%]
tests/replication/test_replication.py::test_tres_replicas_en_nodos_distintos PASSED                                     [ 96%]
tests/replication/test_replication.py::test_bloques_repartidos_entre_varios_nodos PASSED                                [100%]

============================================================ 25 passed in 73.63s (0:01:13) ============================================================
(.venv) ubuntu@ip-172-31-27-218:~/DFSha$
```

9.2 Prueba de carga y recuperación de archivos

Pruebas: test_put_get_20mb, test_archivo_vacio, test_put_sobre_existente_reemplaza y test_bloques_repartidos_entre_varios_nodos. Resultado en AWS: aprobadas. Demostración con la CLI: un archivo de 105 MB (13 bloques de 8 MiB, 39 réplicas repartidas entre los 5 nodos: 8, 8, 8, 8 y 7 bloques) se carga en 8,49 s y se recupera en 4,75 s; el SHA-256 del archivo recuperado es idéntico al del original (capturas en el anexo 14.1).

9.3 Verificación de integridad mediante hash

Prueba en AWS: test_replica_corrupta_en_contenedor_se_descarta (tests/replication/test_integrity_docker.py). Se altera un byte del bloque cifrado dentro del contenedor de un nodo de datos; el tag GCM deja de validar y el cliente, al leer repetidas veces, siempre obtiene el contenido original desde otra réplica. Resultado: aprobada. La variante original (test_replica_corrupta_se_descarta_y_se_lee_otra) solo corre en modo local y se omite en Docker.

9.4 Pruebas de operaciones RF3

Pruebas: test_open_write_close_read, test_reemplazar_crea_nueva_version_y_anexar, test_escritura_no_visible_hasta_close, test_lector_termina_sobre_la_version_que_abrio y test_lectura_por_rango_entre_bloques. Comprueban la semántica declarada: crear, reemplazar y anexar; una escritura no es visible hasta close(); un lector sigue sobre la versión que abrió; read() admite rangos que cruzan bloques. Resultado: aprobadas.

9.5 Pruebas de concurrencia

Pruebas: test_lock_exclusivo_entre_clientes, test_locks_compartidos_conviven, test_escritura_de_otro_cliente_sobre_bloque_bloqueado, test_version_conflict_gana_un_solo_escritor y test_subidas_paralelas. Dos clientes sobre el mismo archivo o bloque: el lock exclusivo excluye, los compartidos conviven y, ante un conflicto de versión, gana un solo escritor. Resultado: aprobadas.

9.6 Pruebas de replicación y caída de nodos

Pruebas: test_lectura_y_escritura_con_un_dn_caido, test_re_replicacion_tras_caida, test_caida_del_lider_de_metadatos y test_tres_replicas_en_nodos_distintos. Con un nodo de datos caído, la lectura y la escritura continúan y el sistema re-replica hasta R=3. Con el líder de metadatos caído, el cliente se recupera y lo confirmado se conserva. Resultado en AWS: las 3 pruebas de fallos aprobadas en 124 s (incluye ~30 s de detección del nodo caído); la de réplicas en nodos distintos, aprobada.

9.7 Pruebas de consistencia de réplicas

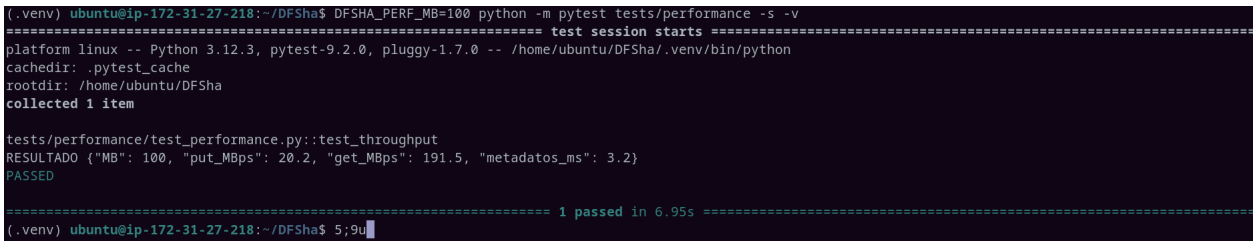
Prueba en AWS: test_replica_desconectada_no_sirve_version_vieja (tests/failures/test_replica_atrasada.py). Se detiene un nodo que tiene una réplica, se reescribe el archivo (la escritura se confirma con W=2), el nodo vuelve a conectarse y se comprueba que todas las réplicas que el servidor de metadatos publica para la versión vigente devuelven el hash de esa versión, y que la lectura entrega el contenido nuevo. Resultado: aprobada. El mecanismo es el block report del nodo al reiniciarse, que el servidor de metadatos contrasta con el estado confirmado.

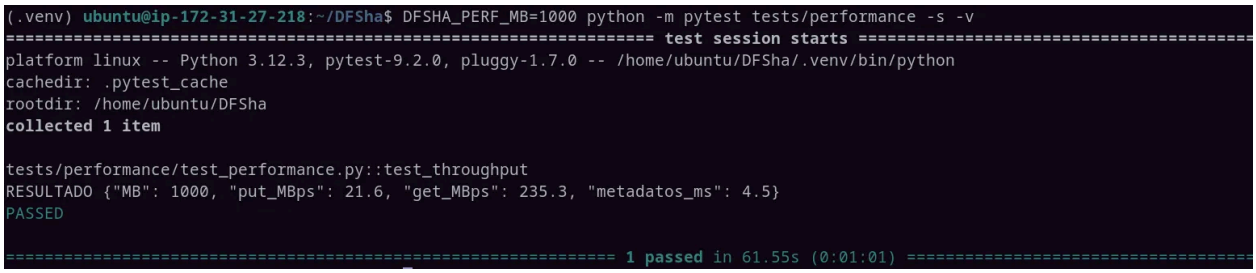
9.8 Pruebas de seguridad: acceso autorizado y denegado

Pruebas en AWS, todas aprobadas: test_aislamiento_entre_usuarios (un usuario no accede a las rutas de otro); test_nodo_de_datos_rechaza_lectura_sin_token_valido (sin token, con token roto o con la firma alterada, el nodo responde PERMISSION_DENIED); test_token_de_otro_bloque_no_sirve; test_bloque_en_disco_esta_cifrado (el texto plano no aparece en el archivo que guarda el nodo); y test_mtls_rechaza_cliente_sin_certificado (un canal TLS sin certificado de cliente y un canal en claro no logran conectar). Además, una captura de red con tcpdump durante la carga de un archivo de 105 MB no contiene el texto plano del archivo (0 apariciones del marcador; capturas en el anexo 14.1).

9.9 Pruebas de rendimiento: latencia y throughput

Prueba test_throughput en la VM de AWS (t2.large, mTLS activo, 5 nodos de datos), con archivos de 100 MB y de 1 GB (1000 MB). Con 100 MB: put 20,2 MB/s, get 191,5 MB/s y latencia de metadatos de 3,2 ms. Con 1 GB: put 21,6 MB/s, get 235,3 MB/s y latencia de metadatos de 4,5 ms; la prueba completa aprobó en 61,55 s.





9.10 Comparación de resultados bajo distintas condiciones

Se cargó un archivo de 1 000 000 000 bytes, que el sistema dividió en 120 bloques. Cada bloque quedó con 3 réplicas en nodos distintos (360 réplicas en total), y la asignación rotó entre los 5 nodos de datos sin concentrarse en ninguno. La carga tomó unos 46 s (21,6 MB/s) y la descarga unos 4,3 s (235,3 MB/s). Capturas en el anexo 14.1.

10. Resultados y análisis

10.1 Resultados funcionales

RF1, RF2 y RF3 se ejercen en las pruebas funcionales, de concurrencia y de replicación, aprobadas en el clúster local (22 pruebas) y en AWS (20 aprobadas y 1 omitida por ser exclusiva del modo local).

10.2 Resultados de distribución y replicación

Los bloques de un archivo se reparten entre varios nodos de datos y cada bloque tiene tres réplicas en nodos distintos (RNF4). Ver pruebas en las secciones 9.2 y 9.6.

10.3 Resultados de disponibilidad y recuperación

Las pruebas de las secciones 9.4, 9.5 y 9.7 muestran que, ante un conflicto de versión, gana un solo escritor. Los locks exclusivos se excluyen entre sí, los compartidos conviven, una escritura no es visible hasta close(), un lector termina sobre la versión que abrió y una réplica atrasada no sirve versiones viejas.

10.4 Resultados de concurrencia y consistencia

Ante un conflicto de versión gana un solo escritor; los locks exclusivos se excluyen entre sí y los compartidos conviven; una escritura no es visible hasta close() y un lector termina sobre la versión que abrió.

10.5 Resultados de seguridad

Las pruebas descritas en la sección 9.8 confirman que, con mTLS activo en AWS, se rechazan los clientes sin certificado y los canales en claro. También confirman que los nodos de datos rechazan lecturas sin token válido o con token de otro bloque, que los bloques en disco están cifrados, que la captura de red de una carga de 105 MB no contiene texto plano y que los usuarios están aislados entre sí. Las limitaciones se describen en la sección 11.1.

10.6 Resultados de rendimiento

Las cifras están en las secciones 9.9 y 9.10. En AWS (t2.large, mTLS, 5 nodos de datos) se midieron put 20,2 MB/s y get 191,5 MB/s con 100 MB, y put 21,6 MB/s y get 235,3 MB/s con 1 GB. El tamaño del archivo no degradó el rendimiento ni la latencia de metadatos (3,2 y 4,5 ms).

El get es entre 9 y 11 veces más rápido que el put. Una explicación probable es que el put cifra y replica cada bloque a tres nodos en cadena y espera dos confirmaciones, mientras que el get lee en paralelo desde varias réplicas. Es una hipótesis, porque no se midió el desglose. Como los 9 servicios comparten 2 vCPU y el disco de una sola VM, estas cifras no son extrapolables a un despliegue en máquinas distintas.

10.7 Análisis de cumplimiento de requisitos

Requisito	**Estado**	**Evidencia**
RF1	Cumplido	test_subidas_paralelas; prueba con archivo de 1 GB; sin pruebas con más nodos
RF2	Cumplido	test_put_get_20mb

RF3	Cumplido con la semántica declarada (crear, reemplazar, anexar; sin edición parcial)	pruebas de archivo abierto y de locks
RNF1 Escalabilidad	Parcial	test_subidas_paralelas; sin pruebas con más nodos ni archivos de 1 GB
RNF2 Alta disponibilidad	Cumplido en local	pruebas de fallos
RNF3 Consistencia	Cumplido	versiones, commit atómico y locks
RNF4 Particionamiento	Cumplido	test_bloques_repartidos_entre_varios_nodos
RNF5 Rendimiento	Medido en local y en AWS	test_throughput
RNF6 Seguridad	Parcial	ver sección 11
RNF7 Transparencia	Cumplido	el cliente solo conoce los servidores de metadatos

11. Limitaciones y trabajo futuro

11.1 Limitaciones de la implementación

- Seguridad de Raft y claves: el tráfico Raft usa la contraseña de PySyncObj, no mTLS; no hay KMS (las claves maestras llegan por variables de entorno) ni cifrado extremo a extremo (los nodos descifran el bloque para servirlo). Ver sección 7.9.
- El JWT es simétrico (HS256): quien tenga JWT_SECRET puede emitir tokens.

- El registro de usuarios (Register) está abierto a quien llegue al servidor de metadatos.
- Despliegue en una sola VM EC2 t2.large (2 vCPU, 8 GiB de RAM y 30 GiB de disco) con los servicios en contenedores sobre una red bridge de Docker: la comunicación entre nodos es por red real (verificada con tcpdump), pero no entre máquinas distintas. Los 9 servicios comparten CPU y disco, lo que limita el rendimiento medido.
- El estado de los nodos de datos vive en la memoria del líder; tras una elección, una asignación puede fallar brevemente hasta que lleguen los heartbeats.
- Si falta el archivo .env, docker-compose.yml usa valores de desarrollo por defecto (secretos y claves de ejemplo).

11.3 Mejoras y trabajo futuro

- mTLS para el tráfico Raft.
- KMS o rotación de claves.
- JWT asimétrico.
- Registro de usuarios restringido al administrador.
- Particionar el espacio de nombres por usuario en varios grupos Raft.
- Pruebas con distintos tamaños de bloque y de R, y con nodos en máquinas distintas.
- Edición parcial por bloque con copy-on-write (propuesta del diseño individual de María).

12. Conclusiones

DFSha cumple RF1 a RF3 con una arquitectura S2S de tres servidores de metadatos (Raft) y cinco nodos de datos, con bloques de 8 MiB, tres réplicas y quorum de escritura de dos. Desplegado en AWS con Docker Compose y mTLS, se comprobaron la carga y recuperación de un archivo de 105 MB y de 1 GB, con SHA-256 idéntico, la tolerancia a la caída de un nodo de datos y del líder de metadatos, la exclusión entre escritores, la publicación atómica de versiones, el rechazo de accesos sin token válido y el cifrado en reposo.

Las principales limitaciones son el despliegue en una sola VM de recursos reducidos (sin prueba de 1 GB), Raft sin mTLS y la ausencia de KMS y de cifrado extremo a extremo (sección 11).

13. Anexos

Archivos del repositorio: docker-compose.yml (3 servidores de metadatos, 5 nodos de datos y cliente), shared/constants.py (bloque de 8 MiB, R=3, W=2, heartbeat de 3 s,

SUSPECT a los 10 s y DEAD a los 30 s, lease de 30 s) y .env.example (variables requeridas).

Enlace al repositorio GitHub

https://github.com/SoftwareArchitecture-DistributedSystems/DFSha.git

Video

https://drive.google.com/drive/folders/13a8GnySXSwg1UPHhAsoVI-VaV5U3ieII?usp=sharing

14. Referencias

Apache Hadoop. (s. f.). HDFS Architecture. https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html

Brewer, E. (2012). CAP twelve years later: How the “rules” have changed. Computer, 45(2), 23–29. https://doi.org/10.1109/MC.2012.37

Burrows, M. (2006). The Chubby lock service for loosely-coupled distributed systems. Proceedings of OSDI ’06.

Chandra, T. D., y Toueg, S. (1996). Unreliable failure detectors for reliable distributed systems. Journal of the ACM, 43(2), 225–267. https://doi.org/10.1145/226643.226647

Cooper, B. F., Silberstein, A., Tam, E., Ramakrishnan, R., y Sears, R. (2010). Benchmarking cloud serving systems with YCSB. Proceedings of SoCC ’10, 143–154. https://doi.org/10.1145/1807128.1807152

Das, A., Gupta, I., y Motivala, A. (2002). SWIM: Scalable weakly-consistent infection-style process group membership protocol. Proceedings of DSN 2002.

DeCandia, G., et al. (2007). Dynamo: Amazon’s highly available key-value store. Proceedings of SOSP ’07, 205–220. https://doi.org/10.1145/1294261.1294281

Dworkin, M. (2007). Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC (NIST SP 800-38D). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-38D

Ghemawat, S., Gobioff, H., y Leung, S.-T. (2003). The Google file system. Proceedings of SOSP ’03, 29–43. https://doi.org/10.1145/945445.945450

Gifford, D. K. (1979). Weighted voting for replicated data. Proceedings of SOSP ’79, 150–162. https://doi.org/10.1145/800215.806583

Gray, C. G., y Cheriton, D. R. (1989). Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. Proceedings of SOSP ’89, 202–210. https://doi.org/10.1145/74850.74870

gRPC Authors. (s. f.). Documentación de gRPC. https://grpc.io/docs/

Jones, M., Bradley, J., y Sakimura, N. (2015). JSON Web Token (JWT) (RFC 7519). https://doi.org/10.17487/RFC7519

Mohan, C., et al. (1992). ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. ACM TODS, 17(1), 94–162. https://doi.org/10.1145/128765.128770

Ongaro, D., y Ousterhout, J. (2014). In search of an understandable consensus algorithm (Raft). Proceedings of USENIX ATC.

Rescorla, E. (2018). The Transport Layer Security (TLS) protocol version 1.3 (RFC 8446). https://doi.org/10.17487/RFC8446

Rose, S., Borchert, O., Mitchell, S., y Connelly, S. (2020). Zero trust architecture (NIST SP 800-207). https://doi.org/10.6028/NIST.SP.800-207

Shvachko, K., Kuang, H., Radia, S., y Chansler, R. (2010). The Hadoop Distributed File System. Proceedings of MSST.

Tanenbaum, A. S., y Van Steen, M. Sistemas distribuidos: principios y paradigmas. Texto de referencia del curso.